8 Programaris per al control d'instrumentació i el disseny d'instruments virtuals

A mesura que s'ha anat realitzant el desenvolupament dels diferents capítols del text s'han estudiat els sistemes d'instrumentació per a l'adquisició, els mètodes d'anàlisi de les dades adquirides i les tècniques que permeten realitzar el control del sistema físic sobre el qual s'actua, a partir de la informació obtinguda del procés d'adquisició. Sembla necessari l'ús d'un entorn adequat per gestionar tots els diferents passos requerits en el sistema d'instrumentació i control, i un programari adequat per realitzar la seva programació. En aquest capítol es presenta el programari de control apropiat per aconseguir aquest objectiu. Dues característiques fonamentals són exigibles al programari:

- ha de ser capaç de controlar tot el sistema (instruments GPIB, VXI, comunicació sèrie).
- ha de permetre realitzar un disseny personalitzat de l'instrument. L'instrument resultant s'anomenarà instrument virtual.

Així, el programari de control gestiona els procediments de mesura (enviant les ordres que els instruments han d'executar i adquirint les dades mesurades) i processa les dades procedents dels diferents instruments connectats a l'ordinador personal. Per disminuir el temps de programació necessari i alleugerir les dificultats de desenvolupament de l'aplicació, han estat creades algunes eines de programació orientades al control d'instrumentació. Aquest conjunt d'eines constitueixen el que es denomina programari d'instrumentació virtual.

L'escriptura del programa de control és el punt clau de diferenciació entre un programari d'instrumentació virtual i un altre de tipus general, ja que és un dels capítols més importants en el temps de desenvolupament d'un sistema de mesura. La utilització d'un programari d'àmbit general és un procés tediós i subjecte a errors, perquè afegeix a les dificultats de qualsevol programa les derivades de l'ús d'una varietat d'instruments amb ordres de control diferents i de les interaccions entre ells i el dispositiu sota test; però amb programaris orientats a la instrumentació virtual aquesta tasca esdevé molt més simple a causa de la facilitat de programació i control dels dispositius, i també gràcies a l'extensa varietat de llibreries disponibles.

L'estructura típica d'un programa d'instrumentació virtual consta dels mòduls següents (vegeu la Fig. 8.1):

- Interfície gràfica
- Programa de control
- Adquisició de dades
- Anàlisi de les dades

© los autores, 1998; © Edicions UPC, 1998. Quedan rigurosamente prohibidas, sin la autorización escrita de los titulares del "copyright", bajo las sanciones establecidas en las leyes, la reproducción total o parcial de esta obra por cualquier medio o procedimiento, comprendidos la reprografía y el tratamiento informático, y la distribución de ejemplares de ella mediante alquiler o préstamo públicos, así como la exportación e importación de ejemplares para su distribución y venta fuera del ámbito de la Unión Europea.

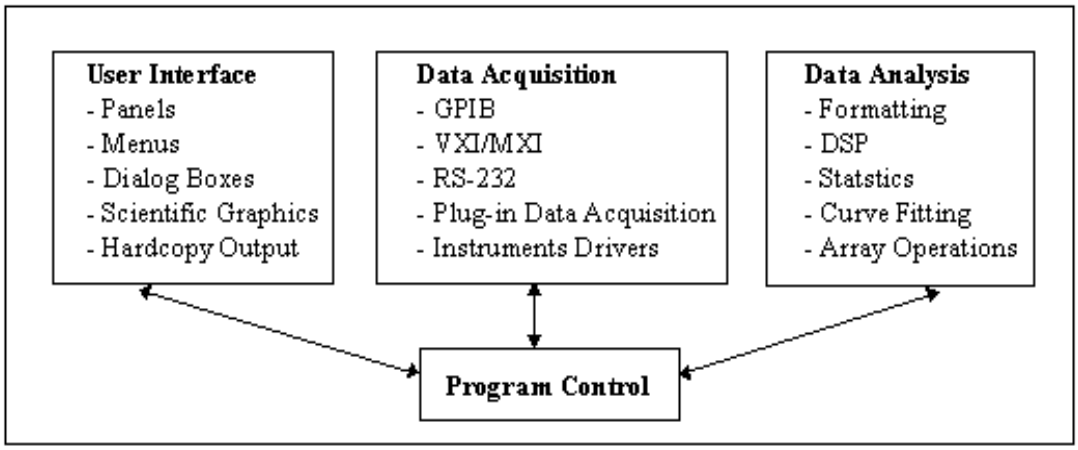


Fig.8.1 Estructura típica d'un programa d'instrumentació virtual

Interfície gràfica (User Interface)

Amb l'editor de la interfície d'usuari es poden construir, amb un esforç mínim, panells interactius per al nostre programa, de tal manera que la interfície gràfica s'adequarà a les nostres necessitats, i gràcies a ella podrem veure tota mena de senyals i dades necessaris per a la interpretació de l'aplicació en curs. Per tot això es disposa d'una sèrie de controls, caixes de diàleg i menús.

Programa de control (Program Control)

El programa de control ens permet controlar tota l'aplicació: l'adquisició de les dades, l'anàlisi d'aquestes i la manipulació de la interfície gràfica. Per realitzar aquesta tasca, els programaris d'instrumentació virtual ens proveeixen de llenguatges d'alt nivell, com poden ser els llenguatges gràfics o el llenguatge C.

Adquisició de dades (Data Acquisition)

A l'hora d'adquirir dades, ja sigui d'un instrument o d'una targeta, és necessari realitzar un control acurat del procés d'adquisició com també de la tasca que ha de realitzar cada dispositiu (instrument o targeta d'adquisició) a cada instant de temps. El programari d'instrumentació virtual permet, mitjançant una sèrie d'ordres, realitzar operacions com ara la lectura o l'escriptura en qualsevol instrument, la targeta d'adquisició, etc.

Anàlisi de dades (Data Analysis)

Després d'adquirir les dades, molt sovint és necessari analitzar-les. Algunes de les opcions que ens proporciona el programari d'instrumentació virtual poden ser el processament del senyal, la realització d'operacions estadístiques sobre les dades, la seva adequació a un tipus de format per poder ser escrites en un fitxer, etc.

© Los autores, 1998; © Edicions UPC, 1998.

És clar que existeixen moltes més possibilitats del programari d'instrumentació virtual que no seran descrites, com poden ser aquelles associades a l'entorn propi del sistema operatiu, és a dir, la comunicació DDE (enllaç dinàmic de dades), la manipulació d'arxius i l'addició de llibreries externes.

En aquest capítol presentarem tres programaris dirigits a la instrumentació, un orientat a comandaments (LabWindows/CVI) i dos més de tipus gràfic (LabVIEW i HP-VEE). En aquesta descripció es parlarà de l'entorn de programació, s'introduiran algunes nocions bàsiques de programació i s'emfatitzarà la comunicació del programari amb la instrumentació.

8.1 LABWINDOWS/CVI

LabWindows/CVI és un sistema de programari per programadors en C. Els programes realitzats en LabWindows/CVI han de respectar les especificacions d'ANSI C, de tal manera que totes les llibreries dinàmiques de dades (DLL's), llibreries en C, *drivers* d'instruments i mòduls objecte han de seguir aquestes especificacions.

La potència de LabWindows/CVI resideix a les seves llibreries:

Per a l'adquisició de dades hi ha cinc llibreries: Instruments Llibreries, GPIB/GPIB 488.2 Library, Data Acquisition Library, RS-232 Library i la VXI Library.

Per a l'anàlisi de dades, hi ha tres llibreries: Formatting and I/O Library, Analysis Library i l'Advanced Analysis Library.

Per a la presentació de dades, ofereix la User Interface Library.

Per a aplicacions de comunicació entre processos, hi ha dues llibreries: la Dynamic Data Exchange (DDE) Library (només per Windows) i la Transmission Control Protocol (TCP) Library.

Totes elles es completen amb una d'ANSI C estàndard.

La Instrument Library és una llibreria especial de LabWindows/CVI. Conté *drivers* per a GPIB, VXI i RS-232 d'instruments com oscil·loscopis, multímetres i generadors de funcions; cada *driver* està generat amb un codi font que pot ser modificat.

8.1.1 Programació amb LabWindows/CVI

Un programa en LabWindows/CVI consta bàsicament de dues parts ben diferenciades: una interfície gràfica que pot ser programada en llenguatge gràfic i un programa escrit en C que gestiona els missatges de control. Com ja hem dit amb anterioritat, la creació d'una interfície gràfica (en aquest cas s'anomena GUI, *Graphical User Interface*) que permeti visualitzar i controlar les operacions que realitza la nostra aplicació serà de gran utilitat. La interfície pot constar de panells, gràfics, polsadors i altres tipus de controls i indicadors, que poden definir-se com:

© Los autores, 1998; © Edicions UPC, 1998.

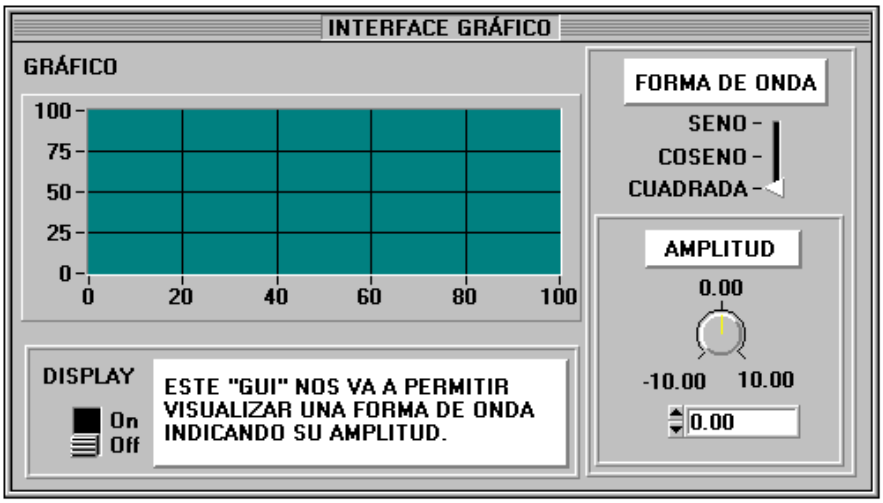


Fig. 8.2 Exemple d'interfície gràfica creada amb LabWindows/CVI

- *Panells*: El primer pas a l'hora de crear una GUI és generar un panell al qual afegirem tots els controls que facin falta. Els panells operen com a *windows* o finestres. Es poden minimitzar, moure, obrir múltiples panells o poden contenir altres panells.
- *Menús*:Una vegada creat el nostre panell, li podem afegir una barra de menús. Aquesta barra sortirà a la part superior del panell i contindrà una sèrie d'opcions que ens poden interessar.
- *Controls*: Un control és un objecte que resideix en un panell accessible per l'usuari i que permet visualitzar informació. La llibreria d'interfície gràfica disposa dels tipus de controls següents: numèrics, botons de comandament, *strings*, LED, gràfics, etc. Cada control té un tipus de dada associada a ell. El tipus de dada d'un control determina el tipus de dades de les variables que s'han d'usar per col·locar o obtenir-ne informació.

La Fig. 8.2 ens mostra un exemple de GUI.

Es pot construir una GUI en LabWindows/CVI interactivament, utilitzant l'editor d'interfície d'usuari (*User Interface Editor*) sense haver d'escriure una línia de codi font. Una vegada acabada la realització del GUI, aquest es gravarà amb un format .uir.

User Interface Events

Quan es crea una interfície, estem definint àrees a la pantalla de l'ordinador, en forma de controls, que poden generar events. Aquests són missatges que genera el programa en treballar en un entorn Windows i que el nostre programa de control pot detectar. Per exemple, quan prenem un *command button* (un polsador) el botó genera un event que és passat al nostre programa escrit en llenguatge C. Els events que es generen en realitzar aquesta operació són els següents:

- GOT_FOCUS event - Si el *command button* no està actiu, en clicar-lo amb el ratolí es genera l'event Got_Focus, i ara es considera que el *command button* està actiu.

© Los autores, 1998; © Edicions UPC, 1998.

- LEFT_CLICK event - En prémer el botó esquerre del ratolí, es genera aquest event. LabWindows/CVI pot discernir entre polsació esquerra, dreta, simple o doble del ratolí.
- COMMIT event - Quan l'usuari deixa de prémer el ratolí, es genera un *COMMIT event* que indica que l'usuari ha realitzat una acció sobre el control.

Cada tipus de control de l'editor de LabWindows/CVI visualitza diferents tipus d'informació i passa diferents events al programa en C. Gràcies a aquests events, ens serà fàcil controlar la nostra GUI ja que en tot moment podem saber quin control ha estat modificat.

Control de la interfície gràfica (GUI)

Una vegada creada la GUI, hem de desenvolupar un programa en C que processi els events generats des de la interfície gràfica. LabWindows/CVI ofereix dos mètodes bàsics per dissenyar programes: l'ús de funcions *callback* i l'ús *d'event-loops*. Quan són usades les funcions *callback*, aquestes són escrites en el nostre programa com a funcions individuals que són cridades directament pels controls de la interfície gràfica. Per exemple, si tenim una funció anomenada ObtenirDades, la qual està assignada com a funció *callback* al *command button* Obtenir de la nostra interfície gràfica, en prémer aquest botó tota la informació d'events que es generi passarà directament a la funció ObtenirDades del programa, a on es processaran. Un exemple de com està construïda la funció en el programa font es mostra a la Fig. 8.3.

En aquest exemple, si es prem el botó Obtenir de la GUI el programa font anirà a la funció ObtenirDades i si l'event que ha estat generat és un EVENT_COMMIT s'executaran les instruccions que estan entre claus. En cas contrari, tornarà al lloc des d'on es va realitzar la crida.

Alternativament, es poden utilitzar *event-loops* per processar els *commit events* que es produeixen. A través de la funció *GetUserEvent* podem identificar quin control genera l'event i el nostre programa podrà processar-lo correctament. El mètode de funcionament és el següent: quan el programa en C arriba a la instrucció *GetUserEvent*, es para a l'espera que es produeixi algun event en la GUI; en el moment en què es produeix, continua endavant. A la Fig. 8.4 tenim un exemple senzill: la funció *GetUserEvent* guarda en una variable el valor de l'event que s'ha produït, en aquest cas, a la variable Ctrl. Si l'event que es produeix és PANNELL_SORTIR, el programa finalitzarà ja que posarà la variable quit=1. En cas contrari l'event que es genera és PANNELL_DIBUIXAR i el programa farà un salt cap a la funció anar_dibuixar( ).

```
int ObtenirDades (int panell, int control, int event, void *callbackData,
        int eventData1, int eventData2)
{
        int i;
        if (event = = EVENT_COMMIT){
        PlotY(handle, PANNELL_SENYAL,dades,punts);
        break;
}
   return(0);
}
```

Fig. 8.3 Exemple d'una funció callback

© Los autores, 1998; © Edicions UPC, 1998.

```
quit = 0;
While (quit = = 0)
{
        GetUserEvent(1,&hndl,&ctrl);
        if (ctrl = = PANNELL_SORTIR) quit = 1;
        if (ctrl = = PANNELL_DIBUIXAR) anar_dibuixar( );
}
```

Fig. 8.4 Exemple d'*event-loop*

Tant les funcions *callback* com els *event-loops* tenen els seus avantatges, depenent de l'aplicació que s'ha de realitzar.

Connexió amb el codi font

Per establir la connexió entre fitxers .uir i fitxers font en C hem d'incloure un fitxer capçalera en el codi font. Quan es crea una GUI és necessari assignar informació a cadascun dels controls per què siguin accessibles posteriorment des del programa d'aplicació. A cada control s'assignarà un nom (anomenat *constant name*) dins l'editor gràfic, que serà utilitzat per a funcions de la llibreria d'interfície gràfica per identificar els controls de la GUI. A més d'assignar *constant name*s a cada control de la GUI, els podem assignar funcions *callbacks*.

Una vegada que se salva un fitxer .uir en l'Editor d'interfície gràfica, LabWindows/CVI automàticament crea el seu fitxer capçalera corresponent (.h) amb la mateixa base que el fitxer .uir. El fitxer capçalera declara tots els *constant name*s i totes les funcions *callback* que han estat assignades al fitxer .uir. En incloure el fitxer .h dins el nostre codi font, tots els *constant name*s i les funcions *callback* associades a la nostra GUI seran automàticament declarades i accessibles.

Les relacions entre ambdós fitxers es defineixen en un fitxer projecte (.prj) a on s'afegeixen els fitxers que interessen (.uir, .h, .c).

Exemple de creació d'una interfície gràfica (GUI)

Anem a crear una GUI amb una opció dins una barra de menú, que ens permetrà representar un senyal en un gràfic. Per sortir-ne un *command button* ens donarà aquesta opció. Per controlar la GUI utilitzarem el mètode de funcions *callback*, encara que es podria utilitzar el *d'event-loop*.

Primerament crearem un fitxer projecte (.prj) a on anirem afegint els fitxers que ens interessin (.uir, .h, .c). Després obrirem un nou fitxer .uir i l'anomenarem igual que el nom donat al .prj. Obrirem el nostre panell i li assignarem el *constant name* PANEL; per fer això anirem a l'opció Create de l'Editor i escollirem l'opció Panel i veurem que aquest posteriorment apareix a la pantalla de la Fig. 8.5.

© Los autores, 1998; © Edicions UPC, 1998.

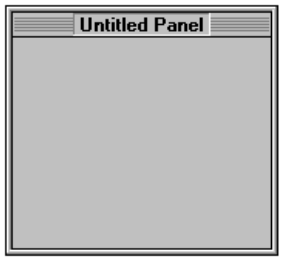


Fig. 8.5 Creació d'un panell

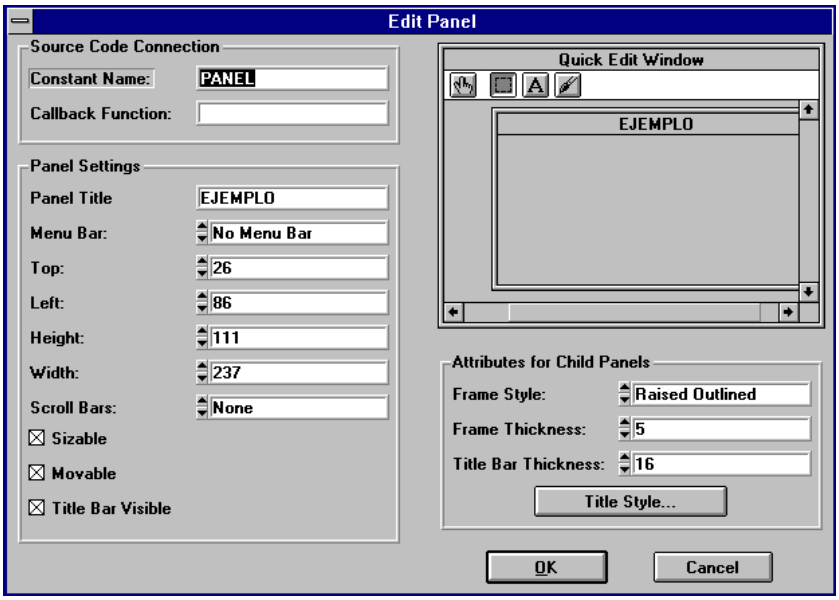


Fig. 8.6 Finestra de control d'un panell

Una vegada tenim el gràfic, hi clicarem dues vegades a sobre amb el ratolí i apareixerà una finestra que permetrà adequar el panell a les nostres necessitats. Assignarem, com ja hem dit, al *constant name* el valor PANEL i li posarem el títol EJEMPLO com ens mostra la Fig. 8.6.

Un cop creat el panell i assignat un *constant name*, el que farem serà incloure-hi els controls necessaris, aquests controls seran un *command button* i un control que permetrà representar gràfics; posteriorment afegirem l'opció de la barra de menú.

Per crear el *command button* podem prémer el botó dret del ratolí i apareixeran totes les opcions de controls que tenim. Anirem a la de *command button* i escollirem una de les formes que ens proporciona. Una vegada seleccionada, la clicarem dues vegades amb el botó dret del ratolí i ens en sortirà la taula de característiques. Li assignarem el *constant name* SALIR i la funció *callback* SalirEjemplo (Fig. 8.7)

© Los autores, 1998; © Edicions UPC, 1998.

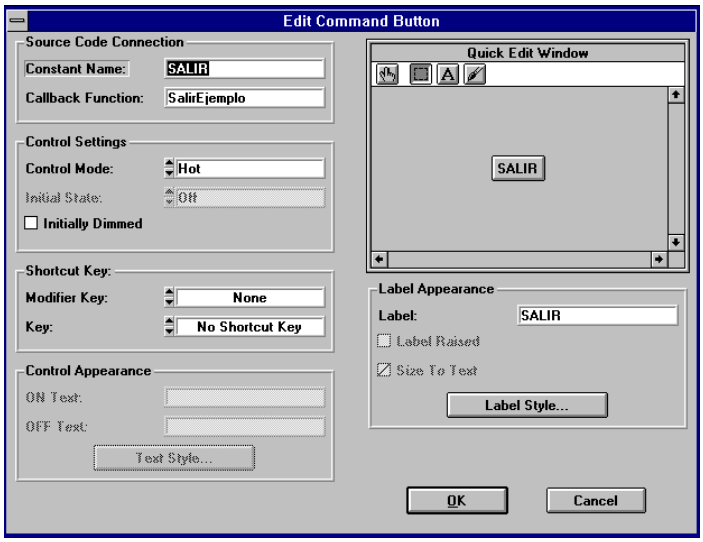


Fig. 8.7 Finestra de control d'un command button

Una vegada creat el *command button*, crearem un control que ens permetrà representar un senyal qualsevol. Per això seleccionarem el control Graph de l'opció Graph, fent el mateix que amb el *command button*, el clicarem dues vegades i ens n'apareixerà la finestra de control. Li assignarem com a *constant name* el valor GRÀFICO; hi apareixeran, a més, una sèrie de característiques, com pot ser el seu mode de control. Com que només ens interessa que representi els valors que se li envien, el posarem com a mode indicador. Una altra opció que ens proporciona és el Data Mode, que ens indica si el gràfic romandrà visible en cas que s'obrin altres finestres. En el nostre cas, no hi ha més panells per obrir, aquesta opció ens és indiferent, així que la deixarem com està. Podem veure-la a la Fig. 8.8.

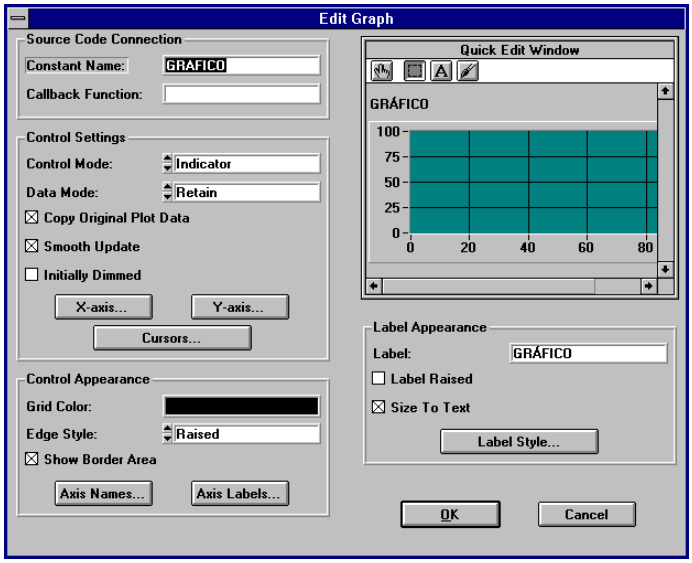


Fig. 8.8 Panell de control d'un control tipus gràfic

© Los autores, 1998; © Edicions UPC, 1998.

Finalment, ens queda per crear la barra de menú, que inclou l'opció de visualització de les dades dins el gràfic. Seleccionarem l'opció Menu Bars... del menú Create. Veiem que apareix una finestra que ens permetrà introduir totes les opcions que volem dins de la barra. En el nostre cas, només ens interessa crear una única opció, que és la de visualitzar les dades en el control gràfic, però pot ser interessant tenir més opcions. A la Fig. 8.9 es mostra un exemple de barra de menú que presenta més d'una opció i a la Fig. 8.10 es mostra com es realitza.

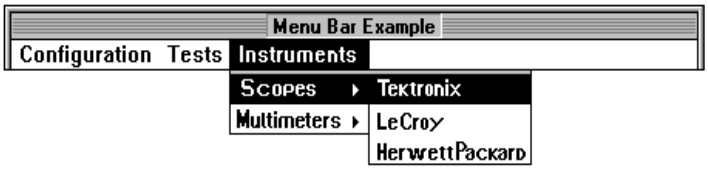


Fig. 8.9 Exemple de barra de menú

Com hem dit abans, només ens interessa una opció a la barra de menú, així que hi assignarem els valors següents:

- Menú Bar Constant Prefix = MENU
- Item = Display
- Constant Name = DISPLAY

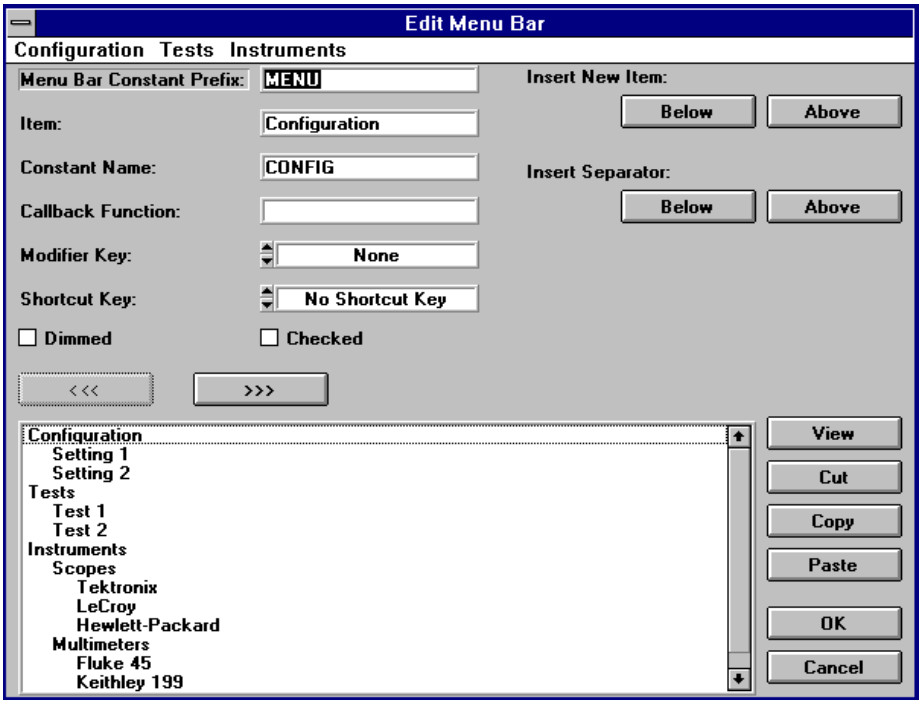


Fig. 8.10 Disseny d'una barra de menú

© Los autores, 1998; © Edicions UPC, 1998.

Una vegada realitzades les operacions anteriors, tenim una GUI perfectament definida i adequada a les nostres necessitats (Fig. 8.11). Vegem ara quin és el fitxer capçalera que ha estat creat a partir dels controls de què disposem. Aquest fitxer s'anomenarà igual que el .uir, però amb extensió .h (Fig. 8.12). Una vegada creats els dos fitxers, els afegirem al fitxer projecte mitjançant l'opció Add File To Project del menú File de l'editor d'interfície gràfica.

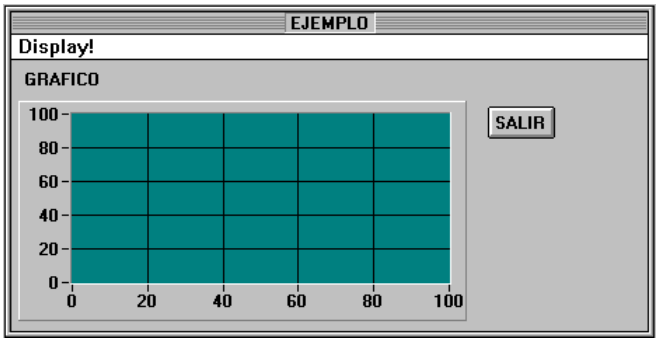


Fig. 8.11 Representació final de la GUI que ha estat creada

Ara només falta crear un fitxer font que realitzi les funcions volgudes i que controli la GUI. Com fins ara, l'anomenarem igual que el fitxer .prj. Primerament declararem els fitxers capçalera de les funcions que es necessitaran. En el nostre cas seran:

- # include "userint.h" : Capçalera de control gràfic.
- # include "analysis.h" : Capçalera de llibreries d'anàlisi.
- # include "exemple.h" : Capçalera que conté els *constant name*s i les funcions *callback* de la nostra GUI.

```
/*****************************************************************************/
/* LabWindows/CVI User Interface Resource (UIR) Include File                 */
/* Copyright (c) National Instruments 1993. All Rights Reserved.             */
/*                                                                           */
/* WARNING: Do not add to, delete from, or otherwise modify the contents     */
/*          of this include file.                                            */
/*****************************************************************************/
    /* Panels and Controls: */
#define   PANEL                   1
#define   PANEL_GRAFICO           2
#define   PANEL_SALIR             3       /* callback function: SalirEjemplo */

    /* Menu Bars, Menús, and Menu Items: */
#define   MENU                    1
#define   MENU_DISPLAY            2
    /* Callback Prototypes: */
int  SalirEjemplo(int panell, int control, int event, void *callbackData, int eventData1,
                  int eventData2);
```

Fig. 8.12 Aspecte del fitxer capçalera associat a la nostra GUI

© Los autores, 1998; © Edicions UPC, 1998.

Una vegada definides passem a presentar la llista del programa.

```
int panel_handle;
main( )
{
        panel_handle=LoadPanel(0,"exemple.uir",PANEL);
        DisplayPanel(panel_handle);
        quit=0;
        while (quit = = 0)
        {
                GetUserEvent(1,&hndl,&ctrl);
                if ( ctrl = = MENU_DISPLAY)
                {
                        SinePattern(100, 1.0, 0.0, 1.0, sine);
                        PlotY(panel_handle, PANNELL_GRAFICO, sine, 100,
                VAL_DOUBLE,VAL_THIN_LINE,VAL_EMPTY_SQUARE,
                VAL_SOLID, 1, VAL_RED);
                }
        }
}
int  SalirEjemplo(int panel, int control, int event, void *callbackData, int eventData1, int eventData2);
        {
                quit = 1;
        }
```

Veiem que el programa no finalitzarà fins que la variable quit = 1. Això ho aconseguirem prement el *command button*. Si no és així, el programa es pararà cada vegada a la funció GetUserEvent en espera que li arribi algun event. En cas que no li arribi, el programa passarà de llarg i tornarà una altra vegada a estar en situació d'espera.

8.1.2 Control d'instruments

Control GPIB

1) Funcions d'obertura i tancament de dispositius connectats al bus GPIB
Primer, per realitzar una transferència d'informació amb el bus GPIB, cal obrir les línies de dades corresponents als instruments que s'han d'utilitzar i assignar-los un valor en el programa per poder ser controlats. Per realitzar aquest procés hi ha dos mètodes.
Primer mètode:

Utilitza la llibreria ibfind. L'estructura d'aquesta llibreria és:

short Board/*Device* = ibfind (char Board/*Device*_Name[ ]);

Si l'executem ens obrirà el dispositiu que hi ha per defecte o un preconfigurat i ens retornarà la unitat que el descriu. Board/*Device*_Name és un *string* que conté el nom del dispositiu i s'assigna mitjançant WIBCONF o ibconf. Però no és una situació normal que l'usuari conegui el nom que l'instrument té.

Segon mètode:

© Los autores, 1998; © Edicions UPC, 1998.

Aquest mètode utilitzarà dues llibreries. Una d'elles és OpenDev, que ens permetrà obrir tots els dispositius accessibles per GPIB. L'altra és ibpad, que assignarà una direcció de bus al dispositiu que és obert. Una condició més comuna és que els dispositius, quan es posen en marxa, indiquin quina és la seva adreça de connexió amb el bus GPIB. Analitzem cadascuna d'elles:

a) OpenDev:
La seva estructura és
int *Device* = OpenDev (char *Device*_Name[], char Instrument_Prefix []);

Igual que en la funció ibfind, *Device*_Name és un *string* que especifica el nom del dispositiu. Si *Device*_Name no és " ", OpenDev actua igual que ibfind. Si, al contrari, *Device*_Name és "", OpenDev actua igual que la funció ibdev que serveix per trobar un dispositiu que no és obert. Instrument_Prefix és un *string* que especifica el prefix de l'instrument associat amb el mòdul d'instrument. Si el mòdul d'instrument no té prefix o OpenDev no és utilitzat en un mòdul d'instrument, es passarà el *string* "" per l'Instrument_Prefix. L'estructura que farem servir serà int *Device* = OpenDev ("","");

b) ibpad:
La seva estructura és
short status = ibpad (short Board/*Device*, short Primary_Address);

S'utilitza per configurar la direcció primària del dispositiu que hem obert. Com es va indicar anteriorment, aquesta adreça ha de estar compresa entre 1 i 30. Un exemple de com s'obrirà i es controlarà un instrument es mostra a la Fig. 8.13. En aquest exemple, el dispositiu controlat té l'adreça 8 del GPIB.

```
int osciloscopio;

osciloscopio= OpenDev( "", "" );        /* obra un dispositiu */
ibpad (osciloscopio, 8 );                /* canvia la adreça de GPIB a 8 */
```

Fig. 8.13 Exemple de connexió amb un dispositiu a través del bus GPIB

Veiem que disposem de dos mètodes per connectar-se amb un instrument. Hem d'anar amb compte i fer servir únicament un dels mètodes en un mateix programa, ja que l'ús de les funcions OpenDev i ibfind en un mateix codi font sovint pot causar conflictes.

Dins de les llibreries d'obertura i control de dispositius, trobarem la funció de tancar la connexió. Aquesta funció és la CloseDev i presenta l'estructura següent:

int Result = CloseDev (int *Device*);

Aquesta funció situa el dispositiu en mode local. *Device* és el descriptor del dispositiu retornat quan l'instrument és obert amb la funció OpenDev. Si CloseDev no pot trobar-lo retorna -1 amb la variable Result. Aquesta funció només es pot utilitzar si el dispositiu ha estat obert amb la funció OpenDev, mai si s'ha obert amb ibfind.

© Los autores, 1998; © Edicions UPC, 1998.

2) Funcions de comunicació amb el bus GPIB
Les funcions més importants, a l'hora de comunicar-nos amb qualsevol dispositiu connectat al bus GPIB, són les de lectura i escriptura. Analitzem cadascuna d'elles.

a) Funció d'escriptura:

L'estructura d'aquesta funció és la següent:

```
short status = ibwrt ( short Board/Device, void *Data/Buffer,
        unsigned long      Byte_Count );
```

Ens permetrà escriure dades des de la memòria fins al dispositiu GPIB. Board/Devide és el descriptor que ens retorna la funció OpenDev o ibfind, Data_Buffer conté les dades que s'enviaran al dispositiu i Byte_Count especifica el nombre total de bytes que es volen enviar.

b) Funció de lectura:

L'estructura és idèntica a la de la funció ibwrt:

```
short status = ibrd (short Board/Device, void *Data/Buffer,
unsigned long Byte_Count );
```

Aquesta funció llegirà del bus GPIB la quantitat de bytes indicats per la variable Byte_Count i situarà totes les dades obtingudes dintre del *buffer* especificat per Data_Buffer. L'operació termina, normalment, quan han estat llegits tots els bytes especificats per Byte_Count o quan es rebi un comandament END. Si l'operació no es realitza en el temps que marca el període de *timeout*, la transferència no queda completada i es produeix un error.

3) Funcions de formatatge i escanneig
Encara que no són funcions de la llibreria de comunicació amb el bus GPIB, aquestes funcions són molt útils a l'hora de treballar amb instruments. Una de les utilitats que presenten és la de convertir les dades que s'obtenen d'un oscil·loscopi o d'un escànner i adequar-les al tipus de format que ens interessa. De la mateixa manera, són útils per crear els comandaments que s'han d'enviar a l'instrument GPIB i no haver de treballar amb la llibreria de funcions de *string*s.

Es poden diferenciar les funcions de formatatge (adeqüen una sèrie de paràmetres de què disposem a una única variable) o les d'escanneig (converteixen una variable obtinguda de lectura de GPIB al tipus de format que es necessita). Expliquem ara cadascuna d'elles.

Funcions de formatatge:
Com s'ha dit abans, les funcions de formatatge serveixen per donar un format nou a una sèrie de dades. Hi ha tres tipus de funcions de formatatge de dades.

Function	Target
Fmt	Memory
FmtFile	File
FmtOut	Standard Out

Funcions d'escanneig:
Les funcions d'escanneig realitzen la tasca inversa a les funcions de formatatge, és a dir, són utilitzades per convertir una única targeta de dades en múltiples fonts. Igual que abans, n'hi ha de tres tipus.

© Los autores, 1998; © Edicions UPC, 1998.

Function	Source
Scan	Memory
ScanIn	Standard In
ScanFile	File

Llibreries i *drivers* d'instruments en LabWindows/CVI

Qualsevol programa que pugui ser carregat amb el menú Instrument es pot carregar com una llibreria d'usuari si es troba el programa prèviament compilat. La principal diferència entre els mòduls carregats com a *drivers* d'un instrument i les llibreries d'usuari és que els *drivers* poden descarregar-se amb el comandament Unload del menú Instrument, mentre que les llibreries no poden ser descarregades. També, com que les llibreries es troben en forma compilada, no poden editar-se mentre es trobin carregades. Les llibreries d'usuari s'instal·len amb el comandament Library Options del menú Options. En entrar en aquesta opció, la secció User Libraries List contindrà els *pathnames* de les llibreries que volem afegir. L'entrada són els *pathnames* dels fitxers de la funció panell (.fp) per al mòdul de la llibreria. La pròxima vegada que es cridi LabWindows/CVI, les llibreries es carregaran automàticament i apareixeran en el menú Library.

La llibreria d'instruments és una llibreria especial del LabWindows/CVI que conté una col·lecció de *drivers* d'instruments. El *driver* d'un instrument consta de quatre fitxers:

El fitxer de gestió de l'instrument, que pot ser .lib, .obj, .dll o .c

El fitxer *include* de l'instrument (.h), que conté la declaració de funcions, la definició de constants i les declaracions externes de variables globals.

El fitxer de la funció panell de l'instrument (.f`p), que conté informació que defineix la funció arbre, la funció panell i el text d'ajuda.

Un fitxer de text en ASCII (.doc), que conté documentació per al *driver* de l'instrument.

Els noms dels quatre fitxers consisteixen en el nom del *driver*, seguit de l'extensió apropiada. Per exemple, si el nom del *driver* és Fft, els noms seran Fft.obj (.c, .lib o .dll), Fft.h, Fft.fp i Fft.doc.

Es poden desenvolupar les diferents parts del *driver* en diverses seqüències:

1. Nom del *driver* de l'instrument.
2. Definir les funcions de l'instrument i les classes de funcions.
3. Crear una funció arbre (que indica la classificació dins del *driver*) per al *driver*, afegir informació d'ajuda en el nivell superior de l'arbre
4. Per a cada funció del *driver*:
   a) Definir els paràmetres de la funció, incloent-hi els tipus de variables i límits, els codis d'error.
   b) Crear la funció panell per la funció del *driver*, incloent informació d'ajuda per al panell i per al control.

© Los autores, 1998; © Edicions UPC, 1998.

c) Afegir algun control secundari que sigui necessari, que inclogui informació d'ajuda.
d) Escriure el codi per realitzar la funció.
e) Testar el codi font.

5. Crear el fitxer *include* pel codi font de l'instrument, que inclogui la declaració de funcions i la definició de constants.

Una cop creat tot el fitxer de la funció panell d'instrument, hem de crear el fitxer *include*:

El primer que cal fer és definir la capçalera del fitxer amb el nom assignat a la llibreria. A continuació es definiran els tipus de variables que s'usen en les funcions i que són d'un tipus predefinit. Finalment, s'escriurà el nom de les funcions amb els seus camps corresponents.

8.2 HPVEE (HP Visual Engineering Environment)

HPVEE és l'aplicació Visual Engineering Environment desenvolupada per HP, que bàsicament és un llenguatge de programació mitjançant icones per resoldre problemes d'enginyeria. Per aconseguir aquest objectiu, HPVEE té un conjunt de llibreries i funcions accessibles per l'usuari que permeten l'adquisició, el control, el processament, la visualització, l'enregistrament i l'actualització de dades, així com el control de rutines lògiques i característiques pròpies de qualsevol llenguatge de programació, sigui C, Pascal o Basic.

Les versions que hi ha del HPVEE són aquestes:
- HPVEE for Windows (PC)
- HPVEE-Test (UNIX)
- HPVEE-Engine (UNIX) (No porta eines de control d'instruments.)
- HPVEE-RunOnly (Executa programes desenvolupats en HPVEE-Test, HPVEE for Windows i HPVEE-Engine.)
- HPVEE-RunOnly for Windows (Executa programes desenvolupats en HPVEE-Test, HPVEE for Windows i HPVEE-Engine.)
- HPVEE for Sun SPARCstation.

Un aspecte que cal destacar del HPVEE és que no té eines per a l'adquisició de dades amb targeta d'adquisició. Existeix però, una versió del HPVEE comercialitzada per *Data Translation, Inc.*, anomenada DTVEE, que afegeix un menú d'adquisició de dades que permet la utilització de targetes d'adquisició de PC per fer mesures automatitzades.

8.2.1 La programació en HPVEE

Menús HPVEE

La finestra principal de HPVEE té un menú en el marge superior format per:
- File. Amb opcions d'obrir, tancar, guardar, etc, fitxers.
- Edit. Amb opcions de copiar, tallar, enganxar, etc.
- Flow. Indica com s'executa el programa.
- Device. Es refereix al tractament amb instruments.

© Los autores, 1998; © Edicions UPC, 1998.

- I/O. Tracta les rutines d'entrada i sortida de dades.
- Data. Es refereix al format de les dades.
- Math i AdvMath. Es refereixen als paràmetres matemàtics.
- Display. Opcions de finestra.
- Help. Ajuda.

Objectes i connexions

La programació en HPVEE resulta molt senzilla i ràpida, ja que només es tracta d'interconnectar les icones (que s'anomenen "objectes", Fig. 8.14) per tal que es passin dades entre elles i executin un procés determinat. Tots els objectes són de forma rectangular i a cada costat tenen un tipus de connexió diferent. A la part superior de cada objecte tenim un sol *pin* de connexió, que representa l'entrada de seqüència. Amb aquesta entrada controlem quan volem que s'executi l'objecte. A la part inferior de l'objecte tenim un sol *pin* de connexió que representa la sortida de seqüència. Amb aquesta sortida controlem que els objectes als quals estigui connectada s'executin *després* que l'objecte. A la part esquerra de l'objecte hi pot haver un *pin*, cap o diversos *pins* d'entrada de dades; i a la part dreta, un *pin*, cap o diversos *pins* de sortida de dades, depenent del tipus d'objecte. La connexió dels *pins* de seqüència és opcional, i només s'utilitza quan és necessari controlar l'ordre d'execució dels objectes.

La connexió entre dos objectes és molt senzilla; només es tracta de clicar amb el botó esquerre del ratolí un dels *pins* que volem connectar i seguidament clicar l'altre *pin*. El VEE s'encarrega d'escollir la ruta que ha de seguir la línia entre els dos *pins* (si tenim aquesta opció seleccionada en les preferències en el menú *File)*. El programa impedeix la connexió incoherent de terminals (connectar dues entrades entre si, per exemple) i "marca" els terminals en què es pot connectar quan s'hi acosta la línia de connexió que apareix en clicar un *pin* per connectar-lo a un altre.

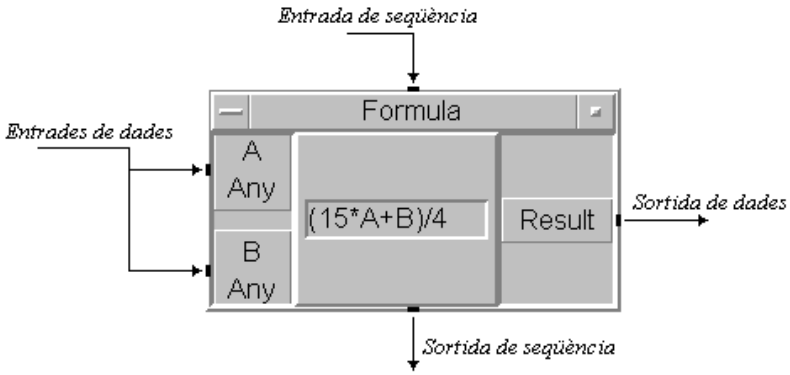
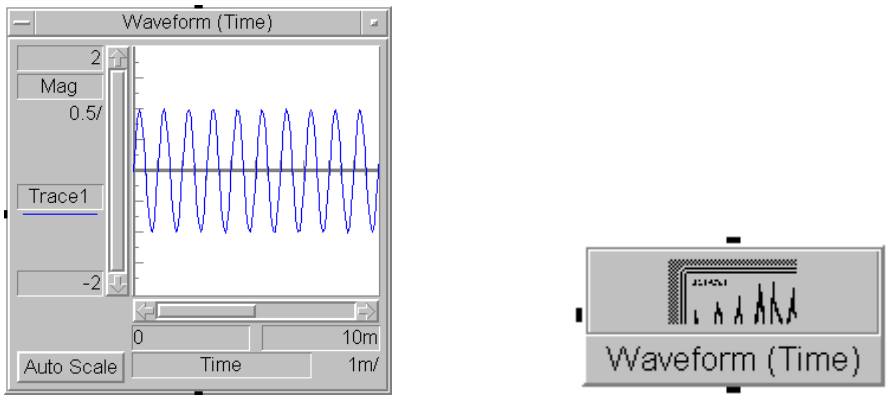


Fig. 8.14 Entrades i sortides d'un objecte

Cada objecte es pot veure de dues maneres: obert o tancat (Fig. 8.15). Un objecte obert ens mostra el seu contingut o l'operació que realitza; un objecte tancat només ens mostra el nom de l'objecte i la seva icona. És recomanable tenir els objectes tancats per guanyar rapidesa d'execució.

© Los autores, 1998; © Edicions UPC, 1998.



(a) (b)

Fig. 8.15 a) Objecte obert. b) Objecte tancat

En els objectes *oberts*, els *pins* passen a ser *terminals* en què es pot veure el seu nom i el tipus de dada que requereixen (Fig. 8.16). Si obrim un terminal de la mateixa manera que obrim un objecte (clicant dues vegades amb el botó esquerre del ratolí), podem veure la dada que conté en aquell moment.

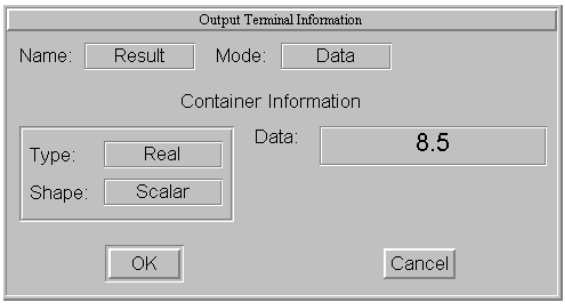


Fig. 8.16 Terminal obert

Cada objecte té el seu propi menú d'edició, on es configuren els paràmetres de l'objecte, com ara el format numèric, el títol, la mida, el color, les escales, etc., depenent de l'objecte que sigui. Aquest menú s'activa clicant amb el botó dret del ratolí qualsevol punt de la barra de títol de l'objecte o les zones dels terminals i també clicant amb el botó esquerre el quadre superior esquerre de l'objecte obert. El menú pot ser diferent per al mateix objecte si aquest està obert o tancat.

User Objects i User Functions

Existeix també l'opció de construir els nostres propis objectes, mitjançant els *User Objects*, als quals afegim les entrades i sortides que volem i hi incloem els objectes que necessitem, i d'aquesta manera reduïm considerablement l'espai ocupat pels objectes. Podríem considerar aquests objectes creats per l'usuari com a "subprogrames" dintre d'un programa principal. Els *User Objects* també es poden crear

© Los autores, 1998; © Edicions UPC, 1998.

a partir d'objectes ja col·locats en el programa principal o en un altre *User Object* (es poden posar *User Objects* dintre d'altres *User Objects* sense limitació). Per fer-ho, només cal seleccionar els objectes que volem que hi estiguin inclosos i seleccionar *Create User Object* en el menú *Edit.* El VEE s'encarrega de crear les entrades i sortides necessàries en el nou objecte segons les connexions que existien i de connectar aquestes entrades i sortides tal com estaven abans de crear l'objecte.

Aquests objectes creats per nosaltres es poden convertir en *User Functions*, que poden ser cridades des de qualsevol punt del programa creat. Per crear una *User Function* a partir d'un *User Object* només cal seleccionar *Make User Function* en el menú de l'objecte. Aquest procés és reversible, i una *User Function* pot tornar a convertir-se en *User Object*.

Podem cridar les funcions tantes vegades com vulguem dintre d'un programa; la funció només ocupa la memòria d'una de sola. Així, quan fem un canvi en la funció, aquest canvi afecta totes les crides fetes a la funció. La crida a una funció es pot fer o bé mitjançant l'objecte *Call Function*, o bé en objectes que permetin escriure el nom de la funció, com per exemple el *Formula* o el *Sequencer*, en els quals s'ha d'escriure el nom de la funció seguit de parèntesis i dintre d'aquests els paràmetres inicials (si n'hi ha). Els parèntesis s'han de posar sempre, encara que no hi hagi paràmetres inicials, per diferenciar una funció d'una simple variable.

Es pot crear una llibreria d'*User Functions*, que serà un fitxer amb una sèrie de funcions fetes per l'usuari i que es pot cridar des d'altres fitxers VEE. Per fer això tenim els objectes *Import Library* i *Delete Library*, que ens permeten importar les llibreries, executar les funcions i esborrar-les de la memòria en temps d'execució, i d'aquesta manera s'estalvia la memòria que s'utilitzaria si s'haguessin de tenir les funcions permanents en cada fitxer en què s'haguessin d'utilitzar, a més d'estalviar espai al disc, ja que la funció es gravaria només una vegada.

Els objectes *Call Function*, *Import Library* i *Delete Library* es troben en el submenú *Function* del menú *Device*. L'objecte *User Object* es troba en el menú *Device*. L'opció de convertir un *User Object* en una *User Function* es troba en el menú de l'objecte.

Panel/Detail

Tot el procés de programació el fem en el *Detail View*, el panell on es veuen tots els objectes i les seves connexions, però es pot crear un panell frontal o *Panel View* on només veurem els objectes del *Detail View* que vulguem, normalment oberts i sense connexions. Per crear aquest panell frontal, s'ha d'activar *Add to Panel* al menú *Edit* un cop s'hagin seleccionat els objectes que s'hi volen tenir. Quan els objectes ja són al panell frontal, tot i ser els mateixos objectes que en el *Detail View*, es poden variar les característiques d'aquests, com la mida, les escales, els colors en els gràfics, etc., sense que aquests canvis afectin l'objecte en el *Detail*. En crear aquest panell frontal apareixen dos botons nous a la part superior de la pantalla: són el *Panel* i el *Detail*, amb els quals commutem entre els dos panells. Aquests panells també els podem tenir en els nostres *User Objects*, de manera que podem fer que els panells frontals dels nostres objectes apareguin mentre s'executen, per exemple, per configurar els paràmetres del nostre programa o per mostrar uns resultats. Això s'aconsegueix activant *Show Panel on Exec* en el menú del *User Object*.

Quan es crea un programa, normalment es vol que l'usuari no sigui capaç de modificar-lo fora de les opcions que dóna el programador; en el VEE podem fer que l'usuari no tingui manera d'accedir al *Detail View*, activant *Secure* en el menú *File*. S'ha d'anar amb compte a utilitzar *Secure*, ja que no

© Los autores, 1998; © Edicions UPC, 1998.

existeix el comandament invers ("*Unsecure*"). Per això s'aconsella al programador tenir una còpia completa del programa que hagi fet. En activar el *Secure* ja no es pot accedir al menú dels objectes i, per tant, no s'en poden modificar les característiques ni la posició en el panell.

El *Secure* també es pot aplicar només a un *User Object*, activant-lo en el menú de l'objecte (es dóna l'opció de gravar la versió íntegra de l'objecte abans d'efectuar el *Secure*).

Bitmaps

Es poden importar gràfics al *Panel View* del programa (o d'un *User Object o User Function*) per utilitzar-lo com a fons. Per fer-ho, només cal activar *Select Bitmap* en el menú *Edit*. VEE importa fitxers gràfics de tipus .BMP i .ICN. També es poden importar gràfics en els objectes tancats per utilitzar-los com a icones de l'objecte, activant *Select Bitmap* en el submenú *Layout* del menú de l'objecte (tancat).

Exemples d'objectes d'importància

**** Confirm (OK)***

Un objecte important en els *User Objects* que creem és el *Confirm (OK)* (Fig. 8.17). És un objecte que permet tenir el panell frontal en pantalla fins que no es prem (té forma de botó); l'*User Object* queda en mode de pausa fins que no s'executa l'OK i es continua el programa normalment. Un error usual en utilitzar aquest objecte és connectar la seva entrada de seqüència amb la sortida de seqüència dels objectes als quals fem l'entrada de dades. Fent-ho així els resultats no són els que volem, ja que en lloc de passar les dades introduïdes per l'usuari, les dades que passen són les que tenia l'objecte per defecte, i l'OK només ha actuat de "porta". El que s'ha de fer és connectar la sortida de seqüència (o de dades) de l'OK amb l'entrada de seqüència de l'objecte on introdueix les dades l'usuari. D'aquesta manera s'aconsegueix que quan s'executa l'OK la dada que passa és la introduïda.



Fig. 8.17 Objecte Confirm (OK)

Confirm (OK) es troba en el menú *Flow*.

**** Sequencer***

Un objecte molt important en el VEE per la seva flexibilitat i potència és el *Sequencer* (Fig. 8.18). Aquest objecte implementa un ordre específic d'execució de tests basat en els resultats en temps d'execució. Cadascun d'aquests tests pot ser una *User Function* o una funció compilada en un altre llenguatge (en C, per exemple). El resultat que retorna el test és comparat amb una especificació del

© Los autores, 1998; © Edicions UPC, 1998.

test per determinar si aquest entra dintre dels marges o no. El *Sequencer* utilitza llavors uns indicadors de passa o falla que determinen l'operació següent que cal realitzar. Hi ha sis opcions per determinar l'acció següent, que inclouen: executar el test següent, repetir el mateix test o saltar a un test anterior. També pot demanar a l'usuari que introdueixi una dada que determini quina acció s'ha de dur a terme. Aquest objecte té molts elements per configurar (d'aquí la seva flexibilitat) i requereix un estudi exclusiu per a l'aprenentatge de la seva utilització.

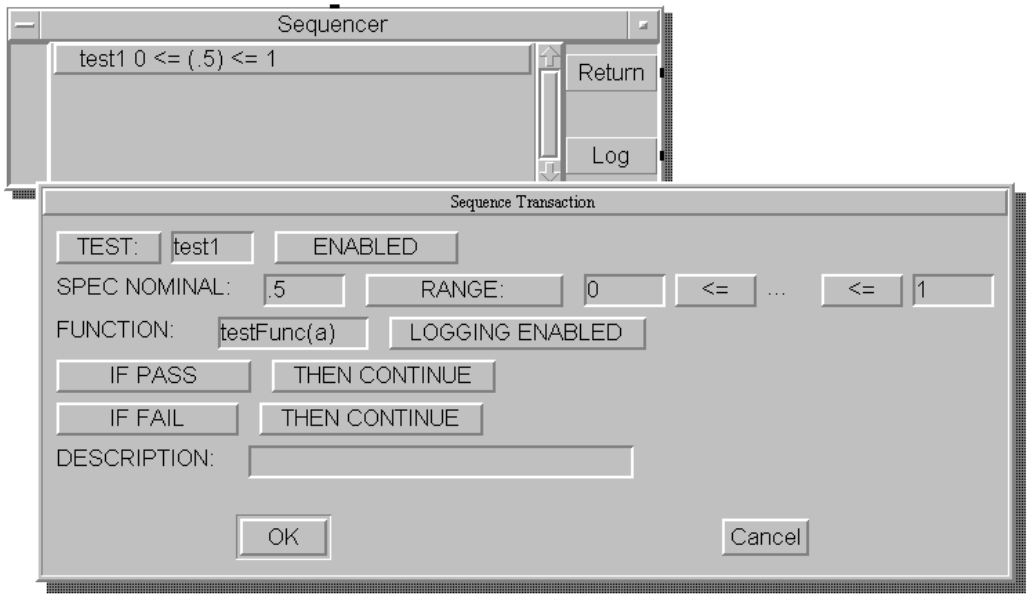


Fig. 8.18 Objecte Sequencer *i configuració d'una acció d'aquest*

Sequencer es troba en el menú *Device.*

**** Formula***

Aquest objecte s'utilitza per escriure expressions matemàtiques o funcions en el HPVEE. Les variables de l'expressió són els noms dels terminals d'entrada de dades, i el resultat de l'execució de l'expressió es col·locarà en el terminal de sortida. Per escriure una expressió a l'objecte *Formula* (Fig. 8.19) s'ha de clicar amb el botó esquerre del ratolí a la zona central de l'objecte (on surt l'expressió *2*A+3* per defecte) i teclejar l'expressió volguda. Per afegir variables (terminals d'entrada), s'ha d'escollir *Add Data Input* en el submenú *Terminals* del menú de l'objecte, o bé prémer les tecles *Control-A* a la vegada que es té el punter del ratolí sobre la zona dels terminals d'entrada (això també és vàlid per a tots els objectes que permetin més entrades de les que tenen per defecte). Un cop es té el nombre d'entrades volgut, es pot canviar el nom d'aquestes, el qual ha de ser conseqüent amb l'utilitzat en l'expressió.

© Los autores, 1998; © Edicions UPC, 1998.

A més d'expressions matemàtiques, a *Formula* també es pot cridar una o diverses funcions, ja siguin creades per l'usuari o del propi VEE. Així, per exemple, podem escriure a *Formula* la funció ABS(X), que dóna el valor absolut de X, en lloc d'agafar l'objecte específic que executa aquesta funció. Això pot ser molt útil quan s'ha d'executar una expressió que inclogui crides a diverses funcions, ja que tindrem un sol objecte per executar tota l'expressió en lloc d'un objecte per a cada funció, cosa que estalvia espai i temps d'execució.

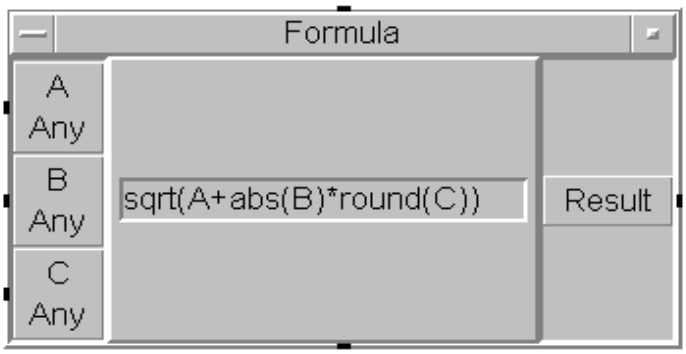


Fig. 8.19 Objecte Formula

Formula es troba en el menú *Math.*

**** Bus I/O Monitor***

Objecte que registra els missatges enviats a través d'una interfície d'entrada/sortida (HP-IB o GPIB, GPIO, VXI o RS-232).

El *Bus I/O Monitor* (Fig. 8.20) registra cada byte (comandaments o dades) transferit entre HPVEE i qualsevol instrument que controli a través d'*Instrument Panels*, *Component Drivers*, *Direct I/O* i els dispositius *Advanced I/O*. Opcionalment la informació que es registra pot ser enviada a un fitxer. L'àrea que mostra la informació conté cinc columnes per a HP-IB, GPIO i RS-232:

- Columna 1: Número de línia.
- Columna 2: Comandament HP-IB (*), dada de sortida (>) o dada d'entrada (<).
- Columna 3: Valor hexadecimal del byte transmès.
- Columna 4: Caràcter ASCII (7 bits) corresponent al byte transmès.
- Columna 5: Mnemònic de comandament de bus (només HP-IB o GPIB)

Per a GPIO només s'utilitzen les tres primeres columnes. Per a RS-232 només s'utilitzen les quatre primeres. Per a VXI la informació que es mostra a les cinc columnes és diferent.

Bus I/O Monitor es troba en el menú *I/O.*

© Los autores, 1998; © Edicions UPC, 1998.

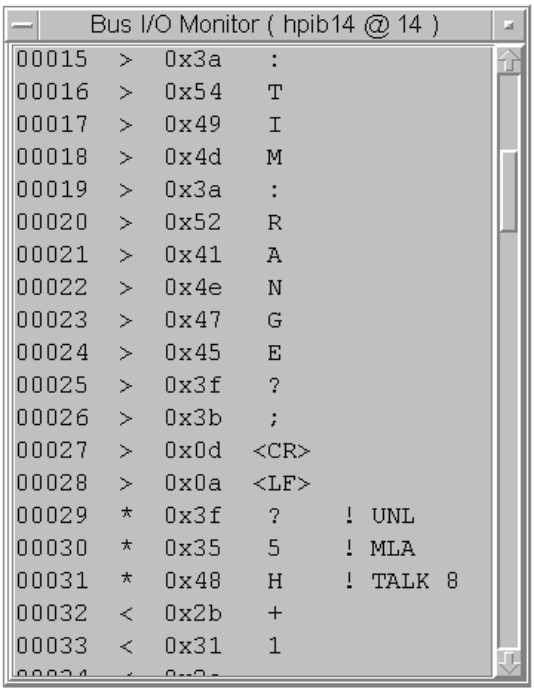


Fig. 8.20 Objecte Bus I/O Monitor

**** Interface Operations***

Objecte (Fig. 8.21) que utilitza transaccions per enviar dades i comandaments d'interfície de baix nivell per via HP-IB i VXI. Ha de ser utilitzat per a tasques específiques d'interfície o per a casos especials en què la comunicació amb els instruments via *Direct I/O* o *Instrument Driver* no aporten el nivell de control necessari.

S'utilitza per a operacions de baix nivell com:

- Enviar seqüències d'adreçament HP-IB no estàndards.

- Enviar un missatge de *trigger* de grup HP-IB o activar línies de *trigger* en VXI.

- Enviar bytes arbitraris via HP-IB, amb ATN cert o sense.

- Enviar comandaments de *clear* i *reset* tant a les interfícies HP-IB com VXI.

- Enviar missatges multilínia definits per IEEE 488.1.

Interface Operations es troba en el submenú *Advanced I/O* del menú *I/O*.

© Los autores, 1998; © Edicions UPC, 1998.

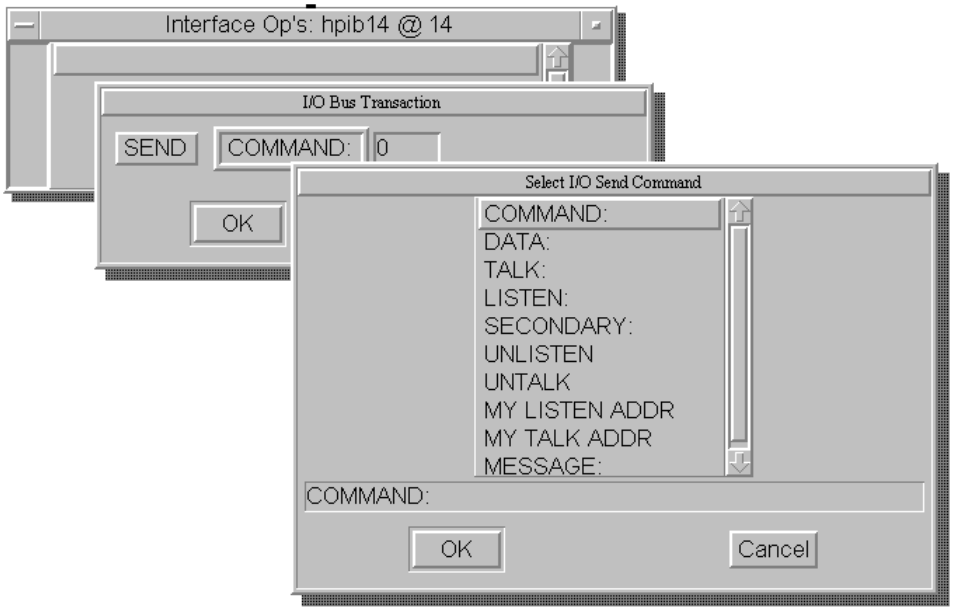


Fig. 8.21 Objecte Interface Operations *i configuració d'una transacció*

8.2.2 Control d'instruments

En HPVEE hi ha dos mètodes per controlar instruments: per *drivers* d'instruments o directament amb l'objecte *Direct I/O*.

Un *driver* és un programa, realitzat en el llenguatge que sigui, mitjançant el qual es pot controlar un instrument sense haver de manejar-lo físicament, sinó a través de l'ordinador, ja sigui per bus HP-IB/GP-IB, per bus VXI, per RS-232, etc., segons l'aparell i el tipus d'interfície de programació que utilitzi. En el HPVEE, l'usuari opera en un panell "virtual" de l'instrument, i el *driver* s'encarrega d'enviar els comandaments corresponents a les accions de l'usuari en aquest panell virtual a l'instrument real.

HPVEE porta una sèrie de *drivers* ja implementats (uns 350 en el programa complet i 23 en el mòdul d'avaluació).

Amb el panell "virtual" de l'instrument no és necessari conèixer els comandaments de programari que controlen un instrument determinat. El que es fa és utilitzar seleccions en menús i finestres de diàleg i el *driver* s'encarrega automàticament d'enviar les cadenes de control correctes a través del bus.

En canvi, amb l'objecte *Direct I/O*, l'usuari ha de saber els comandaments que controlen l'instrument, ja que l'objecte només proveeix un mètode de transmetre comandaments i rebre dades. Amb aquesta opció no és necessari utilitzar *drivers* però cal conèixer els mnemònics propis de l'instrument.

© Los autores, 1998; © Edicions UPC, 1998.

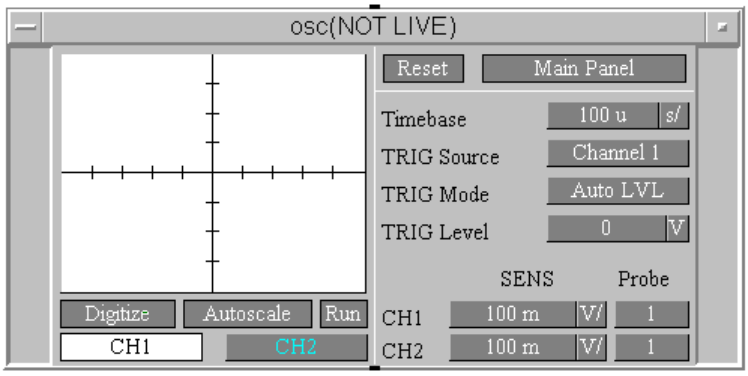


Fig. 8.22 Panell principal del driver *de l'oscil·loscopi HP54600A*

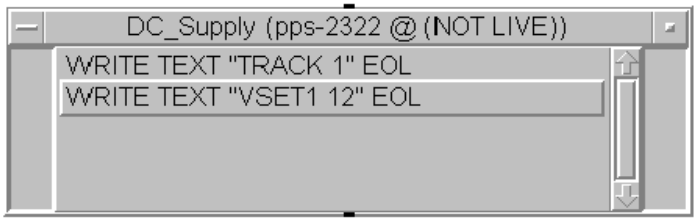


Fig. 8.23 Objecte Direct I/O *de la font de tensió Amrel PPS-2322*

El panell virtual que es veu del *driver* de l'oscil·loscopi (Fig. 8.22) és el principal (*Main Panel*), però n'hi ha nou més, entre ells el *Timebase Panel*, el *Channel Panel* , el *Markers Panel*, etc.

Podem veure a les figures 8.22, 8.23 i 8.24 que al títol hi ha l'expressió NOT LIVE; això significa que l'instrument no està present en el moment de la programació. HPVEE permet construir i executar programes amb instruments encara que aquests no hi siguin o no estiguin connectats. Quan es porta el programa a un altre sistema on sí hi ha l'instrument, o si simplement connectem aquest instrument al nostre sistema, el programa funcionarà correctament (si tenim ben configurat el *driver*). Els avantatges de la utilització del *driver* d'instruments són que és d'ús molt senzill i té una velocitat d'execució acceptable en la majoria dels casos, però com a contrapartida, té el desavantatge de ser útil només per a interfícies HP-IB i que no existeixen *drivers* per a tots els instruments.

Una opció que tenim a l'hora d'utilitzar *drivers* és utilitzar-ne només una part (*Component Driver*, Fig. 8.24), és a dir, només la part que necessitem. En aquest cas, tindrem un objecte "buit" al qual anirem afegint les entrades i sortides que necessitem del *driver*, de la mateixa manera que ho fem amb qualsevol altre objecte. S'ha de tenir en compte que les instruccions que es volen enviar a l'instrument no s'executaran fins que no hi hagi dades en totes les entrades, i que l'ordre d'execució de les entrades i sortides és de dalt a baix. Això és important, per exemple, en configurar l'oscil·loscopi, ja que si posem una entrada que sigui per canviar la base de temps i seguidament una entrada per fer autoescalat, la primera no haurà tingut efecte pràctic.

© Los autores, 1998; © Edicions UPC, 1998.



Fig. 8.24 Component Driver *del generador de funcions Hameg 8130*

Amb els *Component Driver* aconseguim estalviar memòria perquè no tenim un panell virtual com amb el *driver* complet i només tenim les entrades i sortides que necessitem. Malgrat tot, té els mateixos desavantatges que la utilització de *drivers* d'instruments. El *Direct I/O* és més ràpid que els *drivers*, ja que pel bus només hi ha les transaccions que l'usuari envia o demana, sense comprovació d'estat ni d'error, consultes que sí que fan els *drivers*, però, com ja hem dit, no és senzill de programar perquè això requereix el coneixement dels mnemònics de l'instrument.

Si no existeix el *driver* ja fet d'un instrument específic que es necessiti controlar per l'ordinador, tenim les opcions següents:

- Utilitzar l'objecte *Direct I/O* en lloc del *driver*.
- Utilitzar el *Driver Writer's Tool* que ve amb el HPVEE per crear-se el propi *driver*.
- Utilitzar el llenguatge *Instrument Driver Language* per escriure el *driver*

Configuració

L'element *Instrument...* del menú *I/O* activa una finestra (*Instrument Select or Configure*, Fig. 8.25) on es poden afegir, editar o esborrar instruments d'una llista d'instruments configurats, i seleccionar i mostrar els tres tipus d'objectes d'entrada/sortida per a instruments: *Instrument Panels*, *Component Drivers* i *Direct I/O*.

La finestra de diàleg és dividida en tres parts:

•**Llista d'instruments**. Es clica a sobre de l'instrument de la llista que es vol seleccionar. Es clica de nou o es clica el botó *Get Instr* per afegir l'objecte d'entrada/sortida corresponent a l'àrea de treball. El tipus d'objecte dependrà de quin botó estigui seleccionat en l'àrea *Instrument Type*.

•***Instrument Type***. Aquesta àrea determina el tipus d'objecte d'entrada/sortida que s'afegirà a l'àrea de treball quan se seleccioni un instrument. Els tres possibles objectes són: *Instrument Panel*, *Component Driver* i *Direct I/O* (*Instrument Panel* per defecte).

•***Instrument Configure.*** Aquesta àrea permet canviar les configuracions dels instruments. Hi ha tres seleccions possibles:

– ***Add Instrument.*** Aquesta selecció permet afegir un instrument a la llista. Quan se selecciona *Add Instrument*, apareix la finestra de diàleg *Device Configuration*. Es pot configurar l'instrument que es vol afegir a la llista omplint els diferents camps en el *Device Configuration*.

© Los autores, 1998; © Edicions UPC, 1998.

– ***Delete Instrument.*** Aquesta selecció esborra l'instrument actualment remarcat de la llista. (Si s'esborra un instrument que no es volia eliminar, simplement prement *Cancel* es pot recuperar.)

– ***Edit Instrument.*** Aquesta selecció permet editar la configuració d'un instrument existent. En escollir aquesta selecció apareix la finestra *Device Configuration*, on es poden realitzar els canvis necessaris

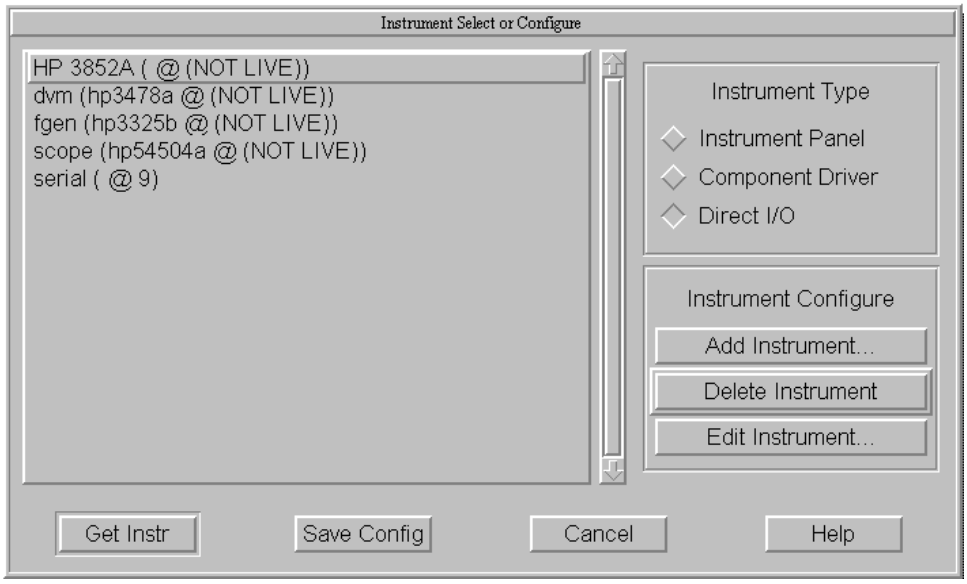


Fig. 8.25 Finestra de configuració-selecció d'instruments

8.3 LabVIEW (Laboratoy Virtual Instrument Engineering Workbench)

LabVIEW, igual que C o Pascal, és un sistema de programació de propòsit general amb extenses llibreries de funcions i subrutines per a qualsevol tasca de programació. Es tracta d'un programari de programació gràfica, on podem trobar les estructures bàsiques per a la programació, com són els bucles WHILE, FOR , estructures CASE, etc. LabVIEW conté també llibreries amb aplicacions específiques per a la gestió de targetes de dades, per al control d'instruments a través de GPIB, VXI i del port sèrie. Permet l'anàlisi, la presentació i l'emmagatzemat de dades. Per a la depuració d'errors inclou eines convencionals de desenvolupament de programes, de manera que podem col·locar punts de ruptura i animar-ne l'execució per veure com les dades passen a través del programa pas a pas.

Destaquem les llibreries següents:

- *File I/O* ⇒ Guarda i recupera dades de suports físics.
- *Analysis* ⇒ *Measurement* → Realitza tota mena de mesures.

© Los autores, 1998; © Edicions UPC, 1998.

Signal Generation→ Simula tota mena de senyals.
Signal Processing→ S'encarrega del tractament de senyals, FFT, convolucions...
Filters→ Conté tota mena de filtres digitals.

- *DAQ* ⇒ Ens permet actuar sobre les targetes d'adquisició.
- *Instrument I/O* ⇒ Per a comunicacions via GPIB, VXI, RS-232...
- *VI* ⇒ Aquesta llibreria permet utilitzar altres programes realitzats prèviament en un altra aplicació.

Els programes realitzats en LabVIEW els anomenem Instruments Virtuals (VI) perquè el seu aspecte i utilització imita instruments reals. Els VI són una interfície interactiva per a l'usuari que accepta paràmetres procedents de VI de nivells superiors; això es concreta de la manera següent:

- La interfície interactiva amb l'usuari d'un VI és el panell frontal, ja que simula el panell frontal d'un instrument físic. El panell frontal pot contenir botons, interruptors, gràfics i altres controls i indicadors (LabVIEW disposa ja d'una sèrie d'objectes predefinits). Les dades són introduïdes amb el ratolí i el teclat, i els resultats són presentats a la pantalla de l'ordinador.

- El VI rep instruccions d'un diagrama de blocs construït en G (*graphical programming language*). El diagrama és una solució gràfica al problema de la programació i és també el codi font per al VI.

- Els VI són jeràrquics i modulars. Es poden utilitzar com a programes principals o com a subprogrames. Un VI inclòs dins un altre VI s'anomena subVI. La icona i el connector d'un VI treballen com una llista gràfica de paràmetres; d'aquesta manera, altres VI poden passar dades a un subVI.

8.3.1 La programació en LabVIEW

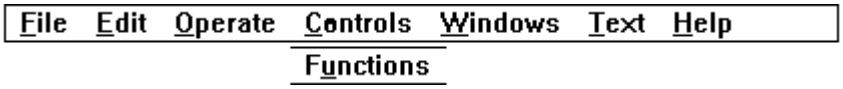
Finestres Pannell i Diagram

Cada VI té dues finestres separades, però relacionades entre si. La finestra *Pannell* conté el panell frontal del nostre VI. A la finestra *Diagram* construïm el diagrama de blocs del programa. Es pot commutar entre ambdues pantalles amb el comandament *Show Pannell/Show Diagram* (Mostrar Panell/Mostrar Diagrama) del menú *Windows* (Finestra). Utilitzant els comandaments *Tile* dins d'aquest menú, podem posicionar les finestres *Pannell i Diagram* una al costat de l'altra o una a sobre de l'altra.

Menús de LabVIEW

La programació en LabVIEW obliga a utilitzar amb freqüència els diferents menús. La barra de menús de la part superior de la finestra d'un VI conté diferents menús *pull-down* (desplegables). Quan cliquem un ítem o element d'aquesta barra, apareix un menú per sota d'ella. Aquest menú conté elements comuns a altres aplicacions Windows, com *Open* (Obrir), *Save* (Guardar) i *Paste* (Enganxar). La figura següent mostra la barra de menús de la versió 3.1 quan la finestra *Pannell* és activa. El menú *Functions* reemplaça el de *Controls* si la finestra *Diagram* és oberta.

© Los autores, 1998; © Edicions UPC, 1998.



A la versió 4



File (Arxiu)	Les seves opcions són utilitzades bàsicament per obrir, tancar, guardar i imprimir VI.
Edit (Edició)	S'utilitza principalment per organitzar el panell frontal i el diagrama de blocs i establir les nostres preferències.
Operate (Funció)	Els seus comandaments serveixen per executar el VI
Project (Projecte)	Presenta els nivells de jerarquia, els subVI que el formen, els que estan sense obrir, busca VI, etc.
Windows (Finestres)	S'utilitza bàsicament per mostrar (*Show*) finestres, com poden ser les d'informació, historial, controls/funcions, eines, portapapers, etc.
Help (Ajuda)	Presenta ajuda sobre les diferents icones i altres aspectes de LabVIEW.

Creació d'objectes

La interfície per a l'usuari d'un VI és la mateixa que la d'un instrument físic, el panell frontal d'un VI té un aspecte similar al de la Fig. 8.26:

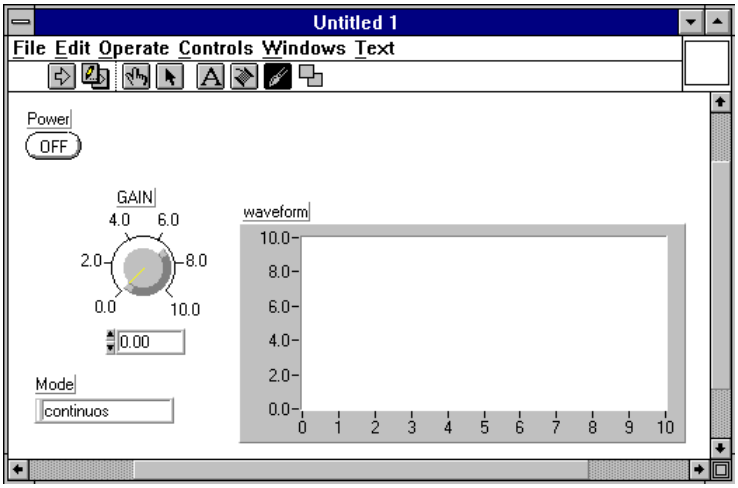


Fig. 8.26 Panell frontal

© Los autores, 1998; © Edicions UPC, 1998.

El panell frontal és principalment una combinació de controls i indicadors. Els controls simulen dispositius d'entrada d'un instrument i proporcionen les dades al diagrama de blocs del VI. Els indicadors simulen els dispositius de sortida de dades de l'instrument que presenten les dades adquirides o generades pel diagrama de blocs.

Es poden afegir controls i indicadors al panell frontal seleccionant-los del menú *Controls* (Fig. 8.27) a la part de dalt de la finestra del panell. Es poden canviar la forma, la grandària i la posició d'un objecte. A més, cada objecte té un menú (*pop-up menu*) des del qual es poden canviar els seus atributs. Accedim a aquest menú prement el botó de la dreta del ratolí sobre l'objecte.

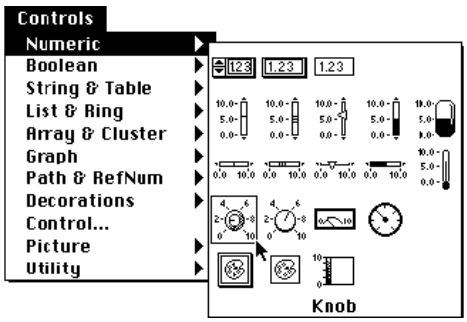


Fig. 8.27 Menú de controls numèrics

L'objecte apareix a la finestra *Pannell* amb un rectangle negre o gris que representa una etiqueta d'identificació o *Label*. Si volem utilitzar-la en aquest mateix moment, introduïm el text des del teclat. Després, qualsevol de les accions següents completa l'entrada:

- Prémer < Shift + Enter >
- Prémer < Enter > del *teclat numèric*.
- Clicar sobre el botó Enter a la paleta d'eines.
- Clicar fora de l'etiqueta.

Quan es crea un objecte sobre el panell frontal, al mateix temps es crea el terminal corresponent sobre el diagrama de blocs. Aquest terminal és utilitzat tant per llegir dades des d'un control com per enviar-les a un indicador.

Si se selecciona *Show Diagram* (Mostrar Diagrama) des del menú *Windows*, podem veure el diagrama corresponent al panell frontal. Aquest diagrama conté terminals per a tots els controls i indicadors del panell frontal.



Fig. 8.28 Col·locació d'etiquetes a controls

A la versió 4.0 el procés en sí que és el mateix. L'única diferència és que no tenim els menús *Controls* i *Functions* a la barra superior, sinó que són finestres flotants que poden ser visibles o no. Si no ho són

© Los autores, 1998; © Edicions UPC, 1998.

utilitzarem l'opció *Show Controls Palette* (Mostrar paleta de controls) o *Show Functions Palette* (Mostrar paleta de funcions) del menú *Windows*.

Una altra opció és clicar amb el botó dret del ratolí qualsevol àrea lliure de la pantalla: ens sortirà el menú *Controls* o *Functions* segons si estem a la finestra *Pannell* o *Diagram*, respectivament.

La finestra de Diagrama conté el diagrama de blocs del VI, el codi gràfic d'un VI de LabVIEW. Queda construït un diagrama de blocs cablejant entre si els objectes que reben o envien dades, desenvolupen una funció determinada i controlen el flux de l'execució.

La figura 8.29 mostra l'exemple d'un VI que calcula la suma i la diferència entre dos números. El diagrama mostra els elements més comuns: nodes, terminals i cables.

Quan se situa un control o un indicador en el panell frontal, LabVIEW situa el terminal corresponent en el diagrama de blocs. No es pot esborrar un terminal que pertany a un control o a un indicador. El terminal desapareix tan sols quan es borra el seu control o indicador.

Les icones de suma i de resta tenen també terminals. Es pot pensar en terminals com a ports d'entrada i sortida. Les dades que són introduïdes en els controls *a* i *b* surten del panell frontal a través dels terminals dels controls en el diagrama de blocs. Les dades llavors entren en les funcions de suma i de resta. Quan les funcions de suma i resta completen els seus càlculs interns, proporcionen els nous valors als terminals de sortida. Les dades flueixen cap als terminals dels indicadors i entren de nou al panell frontal, on són mostrades.

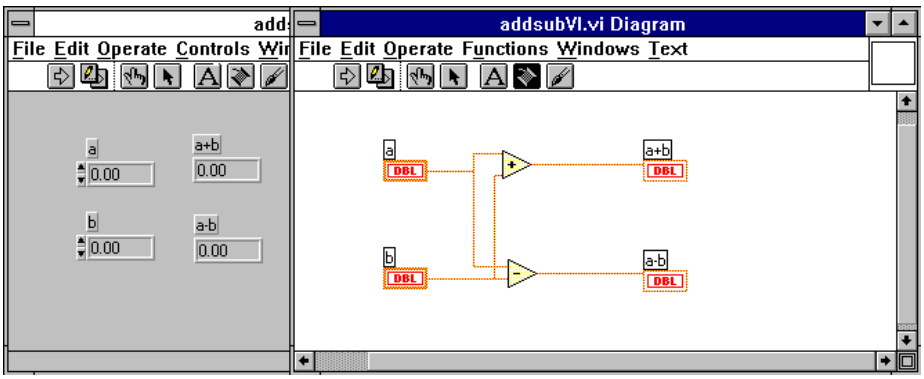


Fig. 8.29 Addsub VI

Els nodes són elements de l'execució del programa. Són anàlegs a les declaracions, operadors, funcions i subrutines en els llenguatges de programació convencionals. Les funcions de suma i resta són un tipus de node. Com ja hem dit, cada panell té associat el seu diagrama, que està programat en un tipus de llenguatge gràfic (anomenat llenguatge G) que es pot considerar com el codi font del programa. En aquest diagrama poden observar-se nodes del tipus bucles For, estructures Case, multiplicacions de funcions i, bàsicament, qualsevol tipus d'estructures de programaris típics com C, Pascal o Basic.

© Los autores, 1998; © Edicions UPC, 1998.

Els cables (*wires*) són els camins de les dades entre els terminals font i els de destí. No es pot cablejar un terminal font amb un altre terminal font, ni un de destí amb un altre de destí, però es pot unir una font amb molts destins. Cada cable té un estil i un color diferents, depenent del tipus de dada que flueix pel cable.

El principi que governa l'execució dels programes de LabVIEW és l'anomenat flux de dades. Un node s'executa únicament quan les dades arriben a tots els seus terminals d'entrada; el node proporciona dades a tots els terminals de sortida en finalitzar la seva execució; i les dades passen immediatament de la font al destí. El flux de dades contrasta amb el mètode de control de flux de les execucions dels programes convencionals, en els quals les instruccions són executades en l'ordre en què estan escrites. El control de flux de l'execució és controlat per les instruccions mentre que el flux de dades és controlat per les mateixes dades. LabView presenta així una capacitat multitasca.

Amb aquestes característiques, LabVIEW promou el concepte de programació modular. Una aplicació és dividida en una sèrie de tasques, les quals poden ser dividides de nou de tal manera que una aplicació complicada pot quedar dividida en una sèrie de més senzilles. Es construeixen VI per complir cada una de les subtasques i després es combinen aquests VI dins un altre diagrama de blocs per complir la tasca principal. Finalment, el VI principal conté una col·lecció de subVI que representen funcions de l'aplicació.

Icones i connectors

Quan un VI actua com un subVI (l'analogia de LabVIEW a una subrutina) dins d'un altre VI, els controls i els indicadors reben i envien dades del VI des del qual s'ha fet la crida.

La icona representa un VI en el diagrama de blocs d'un altre VI. El connector és un joc de terminals que correspon als controls i indicadors del subVI. La icona pot ser una representació pictòrica de la funció que realitza el VI, o pot ser una descripció textual del VI o dels seus terminals.

El connector és similar a una llista de paràmetres de la crida a una funció, els terminals del connector actuen com a paràmetres. Cada terminal correspon a un control o a un indicador particular en el panell frontal. Un connector rep dades en els seus terminals d'entrada i passa les dades al codi del subVI a través dels seus propis controls, o rep els resultats en els seus terminals de sortida procedents dels indicadors del subVI.

Cada VI té una icona per defecte, que es mostra dins la plana del panell (*icon pane*, Fig. 8.30) a la part superior dreta de les finestres del panell i del diagrama. La icona per defecte és una caixa buida tal com es mostra a la il·lustració.

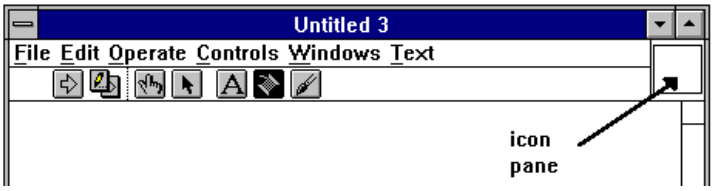


Fig. 8.30 Icon Pane

© Los autores, 1998; © Edicions UPC, 1998.

Cada VI té també un connector al qual s'accedeix triant *Show Connector* del *pop-up menu* de *l'icon pane*. Quan es mostra el connector per primera vegada LabVIEW suggereix un patró pel connector, encara què es pot seleccionar un patró diferent si es vol. El connector generalment té un terminal per a cada control o indicador del panell frontal.

Eines de LabVIEW

Podem crear o canviar un VI quan aquest està en el mode *Edit*. En ell, les eines d'edició s'habiliten a la paleta del mode *Edit,* per sota de la barra del menú de finestra, com s'indica a continuació:



Quan estem a punt per provar el nostre VI, cliquem sobre el botó de mode  o seleccionem *Change to Run Mode* (Canvi al mode d'execució) des del menú *Operate*. Fent això, compilarem el VI i el posarem en el mode *Run*.

Una eina és un mode de funcionament especial del ratolí, i és utilitzada per realitzar funcions específiques d'edició o execució.

- L'eina *Operating*  (Funcionament) maneja els controls del panell frontal (i els indicadors en el mode *Edit).* És l'única eina disponible en el mode *Run*.
- L'eina *Positioning*  (Posicionament) selecciona, mou i redimensiona objectes.
- L'eina *Labeling*  (Etiquetació) crea i edita textos.
- L'eina *Wiring*  (Cablatge) enllaça objectes del diagrama de blocs i assigna als terminals del connector del VI els controls i indicadors del panell frontal.
- L'eina *Coloring*  (Coloració) acoloreix diversos objectes i el fons.

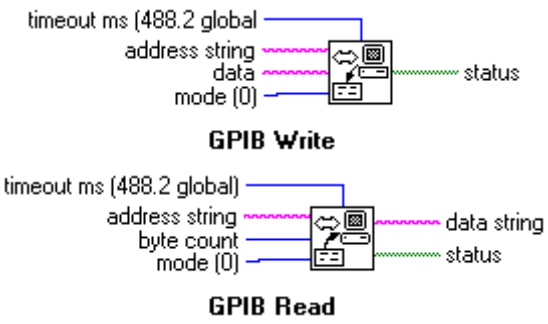
8.3.2 Control d'instruments

Control GPIB

La generació d'un programa de control d'un instrument mitjançant un llenguatge de programació convencional implica una sèrie de problemes, com són dominar tots els comandaments del llenguatge, els comandaments de l'instrument i treballar en un entorn en què la interfície amb l'usuari és poc intuïtiva. Amb LabVIEW gran part d'aquests problemes queden resolts ja que no es necessita escriptura de codi. La part de control i comunicació amb *Bus GPIB* és possible gràcies a una llibreria *GPIB*, mentre que la realització de la interfície ho és gràcies a les llibreries de controls, indicadors, etc.

LabVIEW posseeix una llibreria *GPIB* que ens permet controlar i comunicar-nos amb l'instrument a través del *Bus GPIB.* Les operacions que s'utilitzen amb més freqüència són les de llegir i escriure dades des de o cap a l'instrument mitjançant l'ordinador. Dins de la llibreria *GPIB* de LabVIEW al menú *Functions* hi ha dues icones que s'encarreguen de realitzar aquestes operacions:

© Los autores, 1998; © Edicions UPC, 1998.



Com a paràmetres típics, podem destacar *l'address string*, que indica la direcció de l'instrument GPIB amb el qual volem realitzar la comunicació, i el *status* o *array* booleà, amb el qual s'indica l'estat del controlador GPIB. Canviant aquests paràmetres del VI de la llibreria de LabVIEW es poden efectuar operacions d'initcialització, escriptura, lectura, espera, esborrament, etc., amb instruments GPIB.

Comunicació sèrie

LabVIEW també permet la comunicació pel port sèrie mitjançant la llibreria "Serial"; així es poden prefixar paràmetres de comunicació sèrie com l'entrada i la sortida de dades o la paritat d'aquestes.

Adquisició de dades

El *driver* NI-DAQ és una llibreria dinàmica d'intercanvi de dades en Windows (DLL) que LabVIEW proporciona amb una interfície a nivell alt amb les targes d'adquisició. Amb aquesta aplicació es poden configurar les targes de la llibreria DAQ de LabVIEW d'una manera simple i efectiva, des de NI-DAQ es poden escollir el tipus de senyal que es vol adquirir, el canal, fixar el comptador i el temporitzador, i d'altres possibilitats.

8.4 Exemple: Funció de transferència d'un filtre de pas baix

Construïm un sistema d'adquisició automatitzat en els tres llenguatges, que ens permetrà obtenir funcions de transferència de qualsevol sistema lineal i en particular, d'un filtre de pas baix de primer ordre.

8.4.1 Aplicació en LabWindows/CVI

La figura 8.31 mostra el panell de control de l'instrument virtual creat amb LabWindows/CVI.

Com ja hem comentat, el programa font que controlarà aquesta interfície gràfica ha estat realitzat en llenguatge de programació C. A continuació es mostra un petit fragment d'aquest programa.

```
#include <ansi_c.h>
#include <gpib.h>
#include <utility.h>
```

© Los autores, 1998; © Edicions UPC, 1998.

```
void ir_start(void);
void ir_fft_lab(void);
void ir_dft_real(void);

static double phaseHf_fft_lab[150],magHf_fft_lab[150],array[100];
tatic double array_freq[110],num_array3[2048];
static double num_array[2048],num_array1[2048],cero[2048],cero1[2048];

main()
{
        int quit,hndl,ctrl;
        panell_handle=LoadPanel(0,"scils6.uir",PANNELL);
        quit=0;
        SetCtrlAttribute(panell_handle,PANNELL_MES,ATTR_VISIBLE,0);
        while (quit==0)
        {
                DisplayPanel(panell_handle);
                GetUserEvent(1,&hndl,&ctrl);
                if (ctrl==PANNELL_START) ir_start();
                if (ctrl==PANNELL_LOAD) ir_cargar();
                if (ctrl==PANNELL_SAVE) ir_salvar();
                if (ctrl==PANNELL_QUIT) quit=1;
        }
}
```

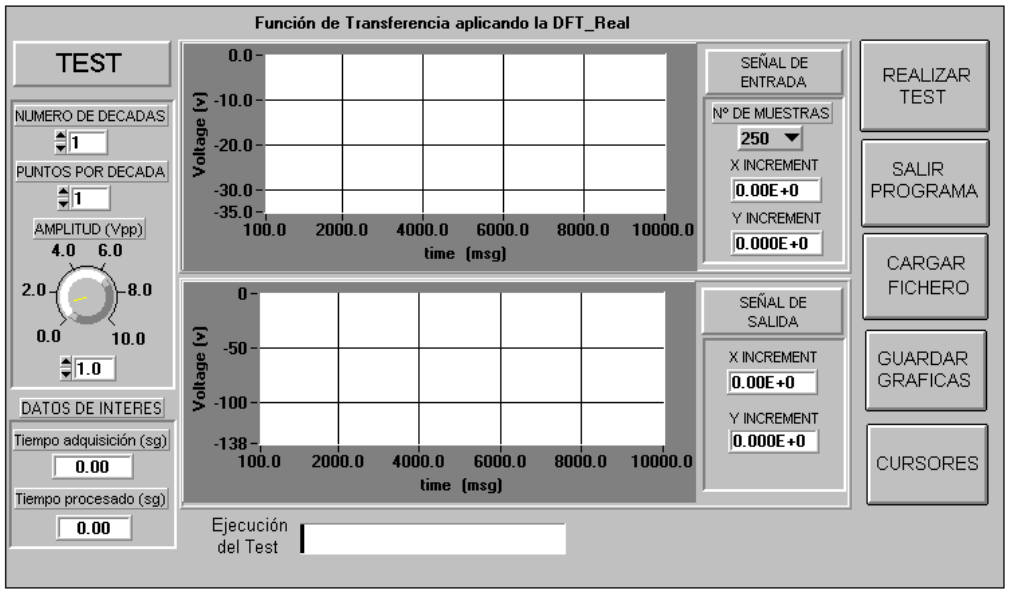
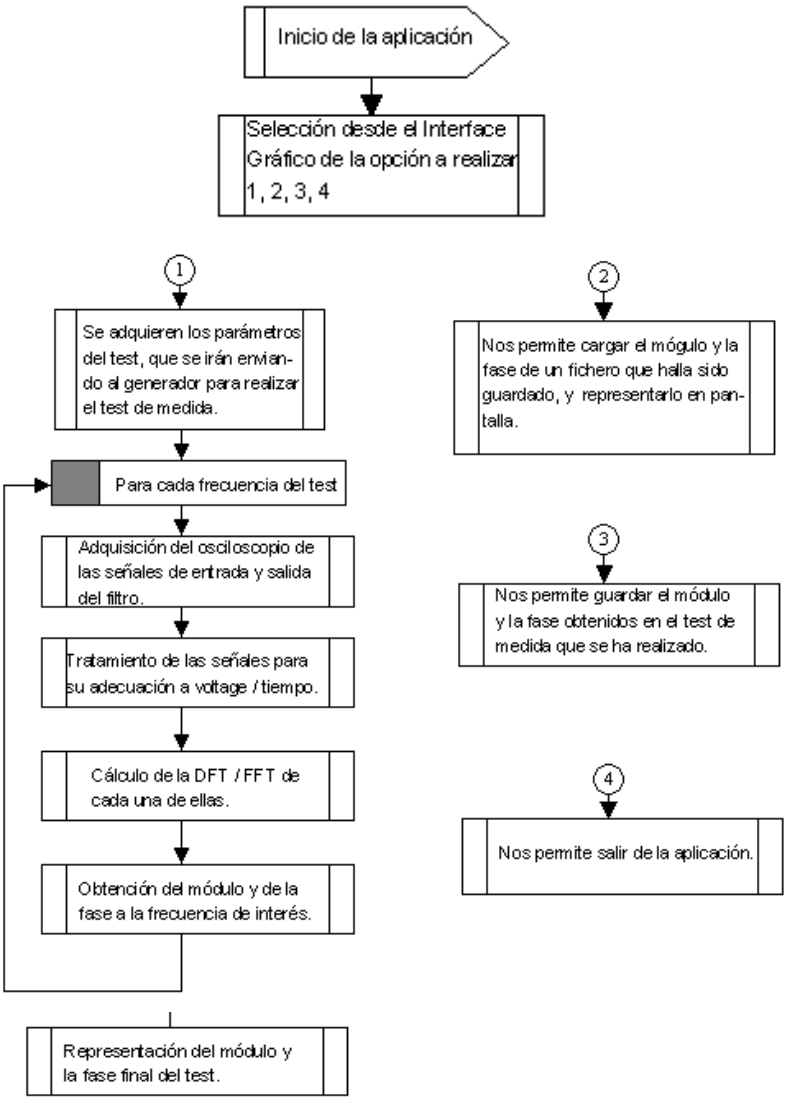


Fig. 8.31 Panell de Control per a l'obtenció de la funció de transferència d'una planta

© Los autores, 1998; © Edicions UPC, 1998.

Veiem que no és més que un programa en llenguatge C, encara que LabWindows/CVI presenta algunes estructures típiques, sobretot per al control de les interfícies gràfiques. El diagrama de blocs de tot el programa de control és, a grans trets, el següent:



8.4.2 Aplicació en HP-VEE

La Fig. 8.32 mostra el panell de control de l'instrument virtual creat amb HP-VEE, mentre que la Fig. 8.33 mostra el panell de programació.

© Los autores, 1998; © Edicions UPC, 1998.

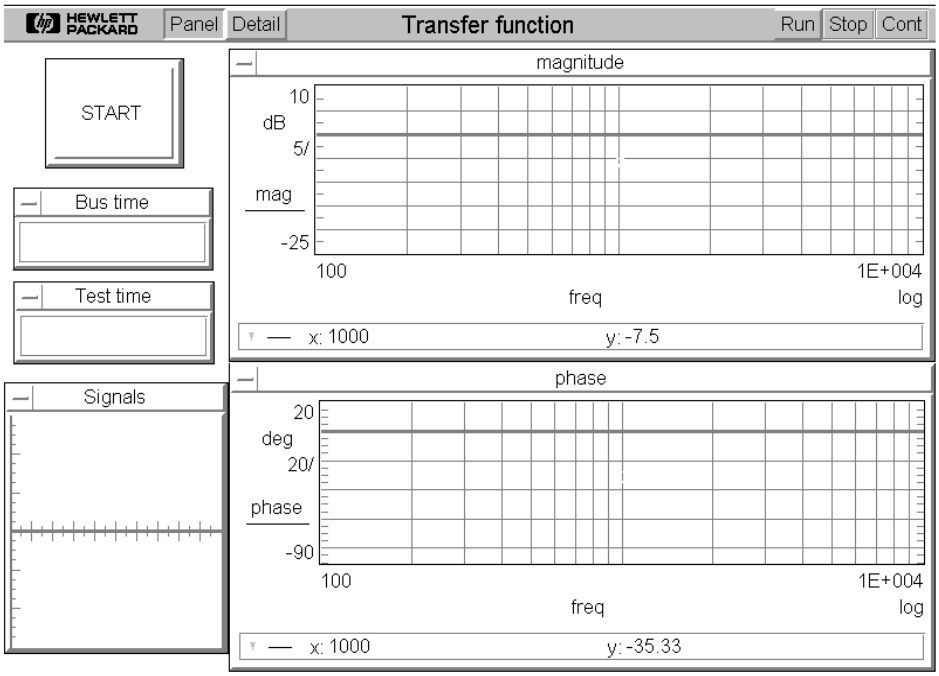


Fig. 8.32 Panell frontal de l'aplicació

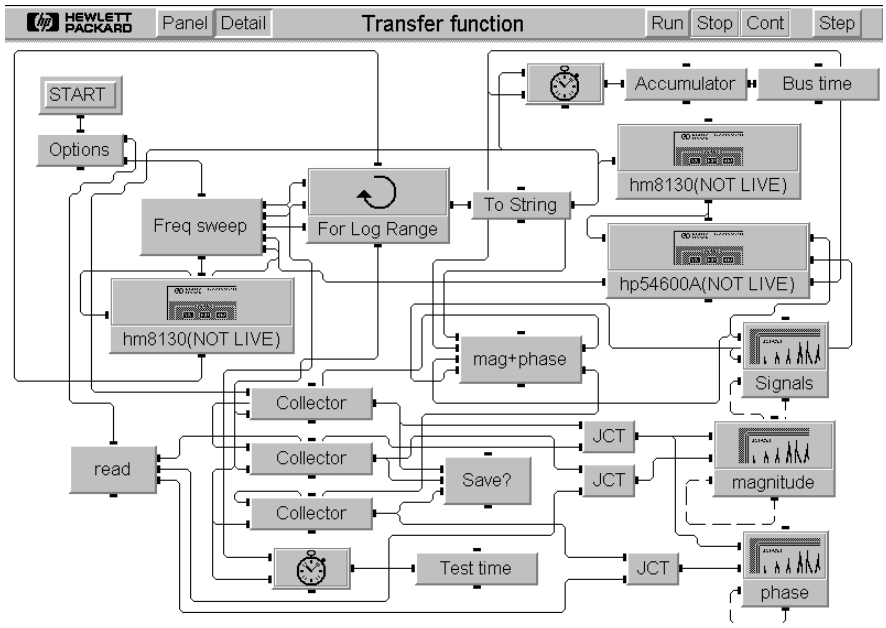


Fig. 8.33 Panell de programació de l'aplicació

© Los autores, 1998; © Edicions UPC, 1998.

8.4.3 Aplicació en LabVIEW

El panell frontal que ha estat creat per controlar l'aplicació es mostra a la Fig. 8.34, i el diagrama de blocs apareix a la Fig. 8.35.

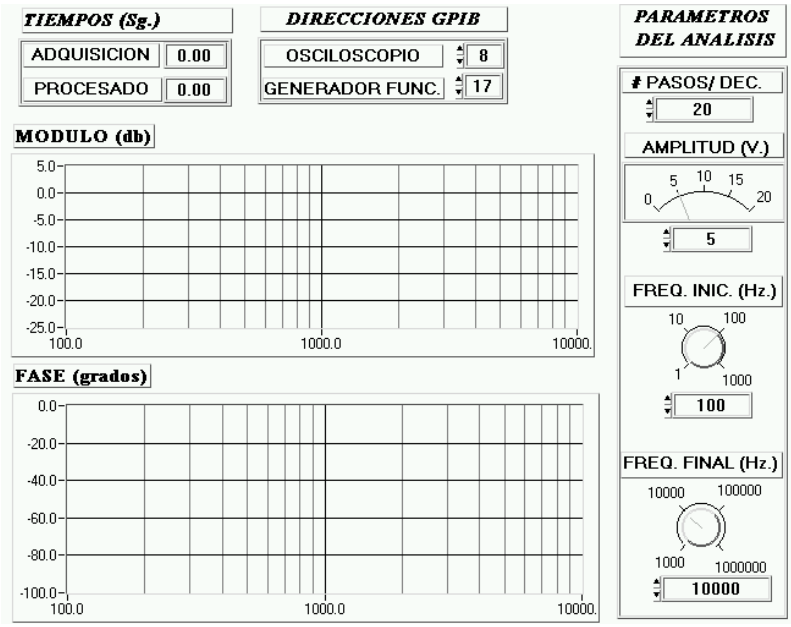


Fig. 8.34 Panell frontal del VI

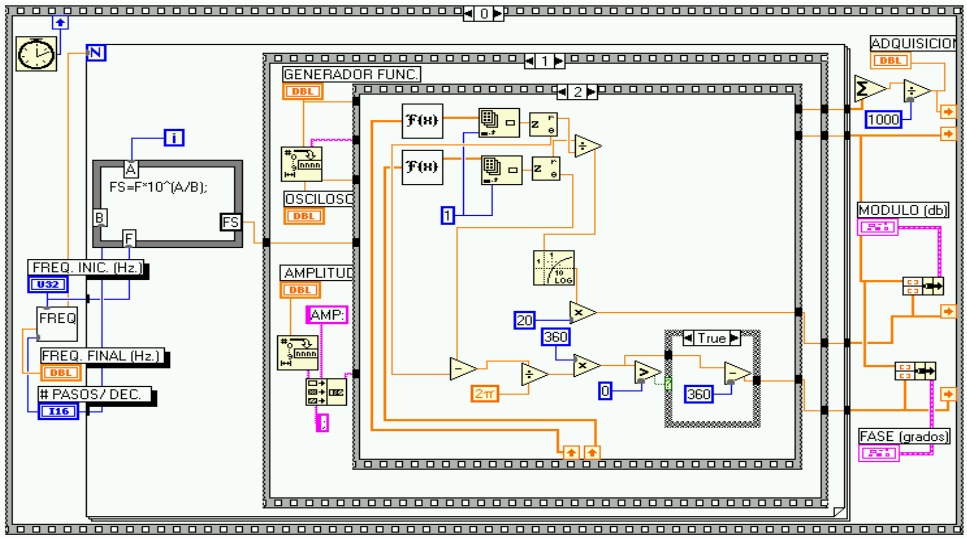


Fig. 8.35 Diagrama de blocs de l'aplicació

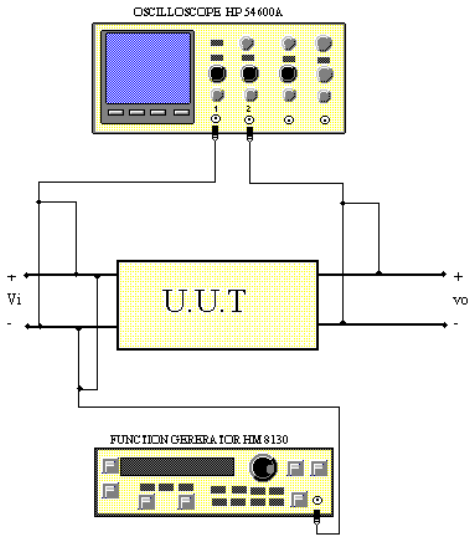
© Los autores, 1998; © Edicions UPC, 1998.

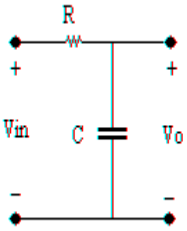
8.4.4 Sistema automàtic de mesura

Les condicions sobre les quals hem desenvolupat el test de mesura són les següents:

Nombre de punts d'adquisició de l'oscil·loscopi: 1.000
Escombratge de freqüències: de 100 Hz a 10 kHz amb 20 punts per dècada
Amplitud del senyal d'entrada: 5 V de pic a pic

L'esquema del sistema de mesura es pot observar a la Fig. 8.36





a) Connexions de l'instrumental de mesura *b)* Dispositiu sota test filtre RC $H(s)=\frac{1}{1+RCs}$

Fig. 8.36 Arquitectura del sistema de mesura

Hem de ressaltar que LabWindows/CVI no presenta dins la seva llibreria d'anàlisi la funció que permet realitzar la DFT. Com que els altres programaris amb què s'ha de comparar fan aquest mètode d'anàlisi, hem creat una DFT en codi C que hem afegit a la llibreria de LabWindows/CVI. Això té una repercussió en el temps d'execució de l'aplicació.

Els dos temps que anem a mesurar al llarg de l'execució de l'aplicació pels tres entorns de programació són el temps de processsament, que no és més que el temps total que s'empra a obtenir el mòdul i la fase de la funció de transferència per a cadascuna de les freqüències, i el temps d'adquisició emprat per l'oscil·loscopi a obtenir els senyals d'entrada i sortida del filtre per a cadascuna de les freqüències d'anàlisi.

© Los autores, 1998; © Edicions UPC, 1998.

8.4.5 Simulació amb MATLAB de la funció de transferència i obtenció de l'error quadràtic mitjà

Calculem ara l'error quadràtic mitjà en mòdul i fase dels resultats obtinguts amb la DFT, respecte a la simulació de la planta en MATLAB. Hem calculat el Bode, simulant en MATLAB el rang de freqüències emprades en el test (Fig. 8.37). D'aquesta manera, una vegada obtinguts els resultats teòrics i pràctics (que han estat importats a MATLAB mitjançant un fitxer de text), es pot calcular l'error quadràtic mitjà com:

$$\sum_{i=0}^{n-1}\left(valor_ideal(i)-valor_real(i)\right)^2 \Big/ n$$

a on:

i= índex de l'array
n= nombre d'elements

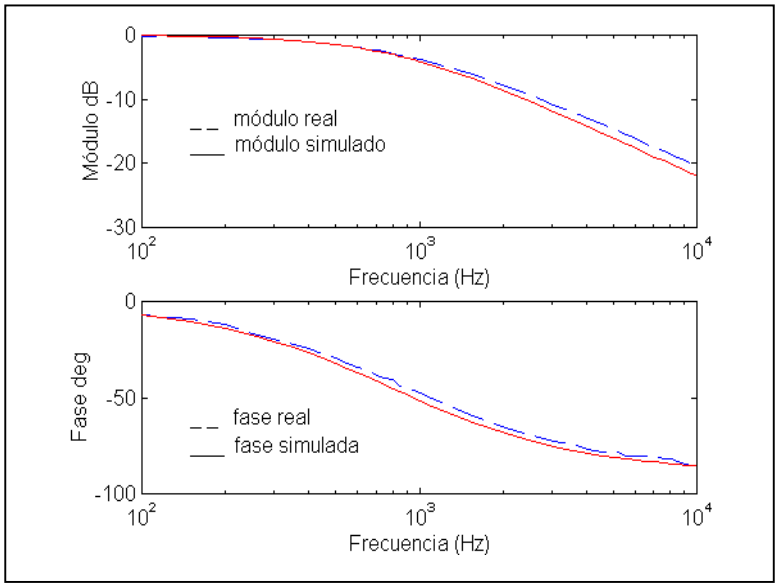


Fig. 8.37 Simulació mitjançant MATLAB del filtre pas baix

Els valors d'error quadràtic mitjà obtinguts són els següents:

	ERROR QUADRÀTIC MITJÀ MÒDUL	**ERROR QUADRÀTIC MITJÀ FASE**
LABVIEW	0,0185	0,3171
LABWINDOWS/CVI	0,0317	0,3530
HP-VEE	0,0035	0,2422

© Los autores, 1998; © Edicions UPC, 1998.

El codi font del programa realitzat per la simulació és el que es mostra a continuació:

```
function filtre(action);
load c:\dft_real.txt
action='initialize';
if strcmp(action,'initialize'),
   figNumber=figure( ...
          'name','Funció de transferència d'un filtre pas baix de 1º orden', ...
      'NumberTitle','off', ...
          'Visible','on');
   axes( ...
      'Units','normalized', ...
      'Psition',[0.05 0.10 0.75 0.95], ...
      'Visible','off');
end;
whitebg(figNumber,'w');
colormap(jet);
num=[1];
den=[200e-6,1];
freq=dft_real(1:40,1);
mòdul=dft_real(1:40,2);
fase=dft_real(1:40,3);
[mag,phase,W]=bode(num,den,freq*(2*pi));
mag=20*log10(mag);
for i=1:40,
          mens_mòdul(i)=(mòdul(i)-mag(i))^2;
          mens_fase(i)=(fase(i)-phase(i))^2;
end;
rms_mòdul=[0];
rms_fase=[0];
for i=1:40,
          rms_mòdul=rms_mòdul+mens_mòdul(i);
          rms_fase=rms_fase+mens_fase(i);
end;
rms_mòdul=rms_mòdul/40
rms_fase=rms_fase/40
subplot (211), semilogx(freq,mòdul,'b--',freq,mag,'r-');
xlabel('Freqüència (Hz)'), ylabel('Mòdul dB');
text(150,-13,'_ _   mòdul real');
text(150,-17,'___  mòdul simulado');
grid off;
subplot (212), semilogx(freq,fase,'b--',freq,phase,'r-');
xlabel('Freqüència (Hz)'), ylabel('Fase deg');
text(150,-60,'_ _   fase real');
text(150,-75,'___  fase simulada');
grid off;
```

8.4.6 Diferències de programació

Pel que fa a la programació, LabWindows, com ja hem dit, basa tota la seva metodologia en la programació en C. Això implica que l'usuari, abans de començar a treballar amb aquest programa, ha

© Los autores, 1998; © Edicions UPC, 1998.

d'adquirir uns coneixements bàsics en aquest tipus de programació. Atès que el llenguatge C és un llenguatge molt utilitzat en els camps de l'electrònica, a l'hora de realitzar implementacions de rutines la seva incorporació a aquest entorn de treball és relativament fàcil. En canvi, LabVIEW presenta una programació gràfica, la qual no requereix inicialment grans coneixements de programació per aprendre'n el funcionament. Malgrat això, els programadors de LabVIEW, si necessiten incorporar llibreries externes al programa, han de realitzar moltes més operacions que les que es necessiten en LabWindows/CVI. A través d'aquestes operacions, es creen els anomenats CIN (*Code Interface Nodes*). Els CIN són programes en C que s'han d'adequar a les necessitats de l'usuari de LabVIEW, i han de ser compilats amb WATCOM, ja que és amb aquest compilador que estan compilades totes les llibreries internes de LabVIEW. D'aquesta manera, si l'usuari necessita crear alguna funció que no és interna a LabVIEW, es veu amb l'obligació d'adquirir coneixements de C.

Agafem com a exemple l'aplicació per a l'obtenció de funcions de transferència a la taula 8.1

Taula 8.1

Programari	**LabWindows/CVI**	**LabVIEW**	**HP-VEE**
Temps d'execució	Temps de bus: 109,03 s Temps processsament:131,4 s TOTAL: 240 s	Temps de bus: 112,37 s Temps processsament:12,36 s TOTAL: 125 s	Temps de bus: 101 s Temps processsament: 70 s TOTAL: 171 s
Error quadràtic mitjà	De mòdul: 0,0317 De fase: 0,3530	De mòdul: 0,0185 De fase: 0,3171	De mòdul: 0,0035 De fase: 0,2422
Grandària de l'aplicació	Fitxer .prj 3.080 bytes Fitxer .uir 20.838 bytes Fitxer .c 12.333 bytes Fitxer .h 2.815 bytes TOTAL : 39.066 bytes	Fitxer .VI 375.381 bytes	Fitxer .vee 92.070 bytes
Tipus de programació	Comandaments C	Gràfica	Gràfica
Programes .EXE	Sí	Sí	No
Ampliació de llibreries externes	ANSI C	CIN (*Code Interface Node*) WATCOM	HP-BASIC
Llibreria de processsament	Àmplia	Àmplia	Limitada

A l'hora de representar gràficament els resultats que s'obtenen de la programació font, LabVIEW presenta considerables avantatges respecte de l'entorn que genera LabWindows/CVI. LabVIEW té en tot moment comunicat el seu codi font (programació gràfica) amb el que han de ser els *displays* que visualitzin les dades, ja que, en generar qualsevol *display*, automàticament es crea la seva variable de control. En tot moment es poden realitzar modificacions sobre qualsevol dels *displays* i aquestes dades les captarà la variable que li hagi estat assignada. A més, els *displays* de representació de gràfics són altament flexibles des de l'execució del programa, com poden ser el canvi de les escales, la posició de cursors, etc., que ens permetran representar els gràfics dintre dels marges del nostre interès. Al

© Los autores, 1998; © Edicions UPC, 1998.

contrari, LabWindows/CVI presenta un major inconvenient en aquest aspecte, ja que el programa font (fitxer .c) es troba separat del programa d'interfície. De la mateixa manera, LabWindows/CVI no permet modificacions en les representacions gràfiques a no ser que es tracti del codi font, és a dir, que si captem un senyal i aquest es troba fora dels marges que ens interessen no podrem realitzar modificacions directament sobre el gràfic, sinó que hauríem d'enviar al programa font els nous marges dels eixos i, des del codi font, enviar una instrucció a la interfície gràfica indicant els nous marges que s'han de posicionar en el gràfic.

Tot al llarg del capítol hem comparat tres eines dissenyades pel control d'instrumentació. Bàsicament ho hem fet a partir d'una aplicació, la caracterització de la funció de transferència d'un filtre de pas baix de primer ordre. Si comparem la grandària de l'aplicació, caldrà triar LabWindows/CVI; si, en canvi, volem velocitat optarem per LabVIEW, i si volem precisió a la mesura a partir de la comparativa d'error quadràtic mitjà, els tres presenten una precisió molt bona, però el HP-VEE és el millor.

Si considerem la facilitat d'aprenentatge i de maneig del programari, ens trobem que per poder treballar amb LabWindows/CVI cal tenir coneixements de llenguatge C. Si considerem les possibilitats de control de dispositius, veiem que HP-VEE no permet la gestió de targetes d'adquisició, ja que cal comptar amb un altre paquet de programes, el DT-VEE.

Finalment podem considerar alguns aspectes subjectius derivats de la nostra experiència de programació. Les llibreries d'adquisició i processat de LabVIEW són molt més extenses i potents que les de la resta de llenguatges, encara que la realització de *drivers* elementals d'instrument són més simples de realitzar en HP-VEE.

Creiem que cada usuari té un perfil molt diferent i segons siguin les seves necessitats pot emprar qualsevol dels tres entorns de control d'instrumentació.

© Los autores, 1998; © Edicions UPC, 1998.

© Los autores, 1998; © Edicions UPC, 1998.